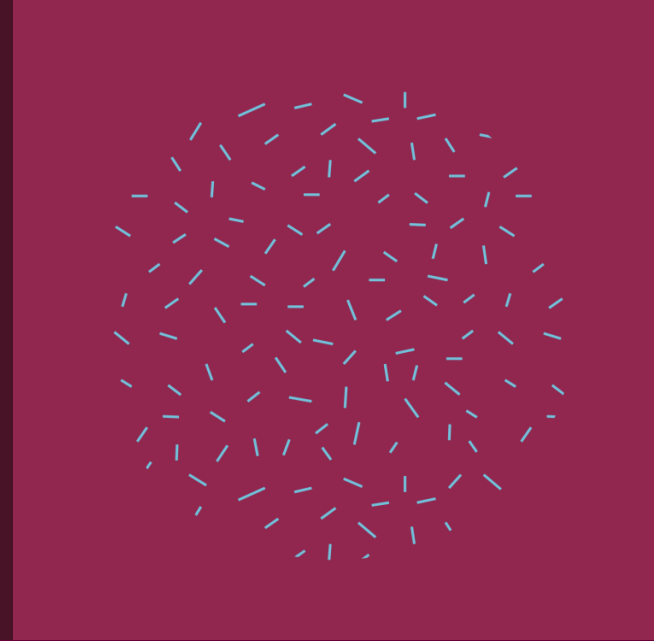




Data Wrangling

Data Visualization , and Preprocessing

Section 2

Eng. Asmaa Fathy

Data Visualization

- Data Visualization is the process of creating graphical representations of data in order to communicate information and insights effectively.
- The goal is to use visual elements such as charts, plots, and maps to make the data more accessible, understandable, and actionable for different audiences.
- Data visualization is a critical aspect of data analysis, and the Pandas library embraces this need by providing built-in functionalities for it.
- It is hard to digest many data at the same time. Data visualization is a way to make the process simple and straightforward. Because data visualization is so important and essential in today's business, Pandas as a data-driven package provides built-in plotting functions.



How the Egyptian economy was from 1980 to 2027 ?

```
import numpy as np
import pandas as pd
```

```
df=pd.read_csv('C:/Users/Admin/Documents/Data Mining Practical/Egypt_Economy_1980_2027.csv')
df
```

	Year	GDP_PPP	GDP_PerCapita_PPP	GDP_Nominal	GDP_PerCapita_Nominal	GDP_Growth	Inflation	Unemployment	Inflation_Change
0	1980	40.3	900.0	22.4	500.0	0.055	0.117	0.096	Nochange
1	1981	56.7	1041.7	31.5	578.7	0.042	0.012	0.095	Decrease
2	1982	73.1	1183.4	40.6	657.4	0.058	0.116	0.090	Increase
3	1983	89.4	1325.1	49.7	736.2	0.075	0.081	0.085	Decrease
4	1984	105.8	1466.8	58.8	814.9	0.040	0.066	0.062	Decrease
5	1985	122.2	1608.5	67.9	893.6	0.040	0.131	0.082	Increase
6	1986	138.6	1750.2	77.0	972.3	0.077	0.152	0.083	Increase
7	1987	155.0	1891.9	86.1	1051.1	0.060	0.147	0.074	Decrease
8	1988	171.3	2033.6	95.2	1129.8	0.036	0.058	0.087	Decrease
9	1989	187.7	2175.3	104.3	1208.5	0.056	0.085	0.098	Increase
10	1990	204.1	2317.0	113.4	1287.2	0.036	0.117	0.128	Increase

Each row represents a year, and each column represents a significant event in the country's economy.

The Data describes three main things:

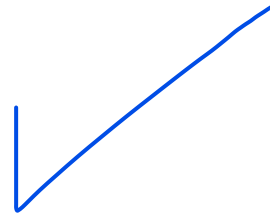
1. How much the country earned?
2. How much prices increased?
3. Are people finding jobs or not?

Imagine Egypt as a family:
Total family income = GDP ,
Per capita income = GDP per capita ,
If income increases = Growth ,
If prices are rising = Inflation ,
If someone can't find a job = Unemployment
This data lets us see if this family was stable?
Or is their income increasing but prices are rising?
Or is unemployment high?

Column	Meaning	Type
Year	It is the year the data represents.	Numeric
GDP_Nominal	It is Egypt's total GDP for that year. Imagine the entire country as one large company. This number tells you how much that company earned this year. The larger the number, the larger the economy.	Numeric
GDP_PPP	It is also the size of Egypt's economy but calculated in a way that takes into account price and cost-of-living differences. Another way to calculate the same size economy.	Numeric
GDP_PerCapita_Nominal	It is everyone's share of the country's economy. That is, if we divide the country's wealth by the number of people, what is each person's share "theoretically"? This doesn't mean that everyone has this amount, but it's an average.	Numeric
GDP_PerCapita_PPP	Same idea but using purchasing power parity.	Numeric
GDP_Growth	It is the most important column to determine: Is the economy improving or not? If the number is: Positive → The country is growing economically. Negative → The country is experiencing problems	Numeric
Inflation	It means how much prices have increased. If inflation is 0.15, it means prices have increased by 15%. Something that used to cost 100 EGP now costs 115 EGP.	Numeric
Unemployment	It is the percentage of people who are unemployed. If it's 0.10, it means 10% of the people who should be working are unemployed.	Numeric
Inflation_Change	It compares inflation to the previous year. Increase → Prices are rising faster. Decrease → Prices are still rising, but more slowly. No Change → Approximately the same rate	Categorical

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 48 entries, 0 to 47  
Data columns (total 9 columns):  
#   Column                Non-Null Count  Dtype    
---  ---  
0   Year                  48 non-null    int64    
1   GDP_PPP               48 non-null    float64  
2   GDP_PerCapita_PPP     48 non-null    float64  
3   GDP_Nominal           48 non-null    float64  
4   GDP_PerCapita_Nominal 48 non-null    float64  
5   GDP_Growth            48 non-null    float64  
6   Inflation             48 non-null    float64  
7   Unemployment          48 non-null    float64  
8   Inflation_Change      48 non-null    object   
dtypes: float64(7), int64(1), object(1)  
memory usage: 3.5+ KB
```

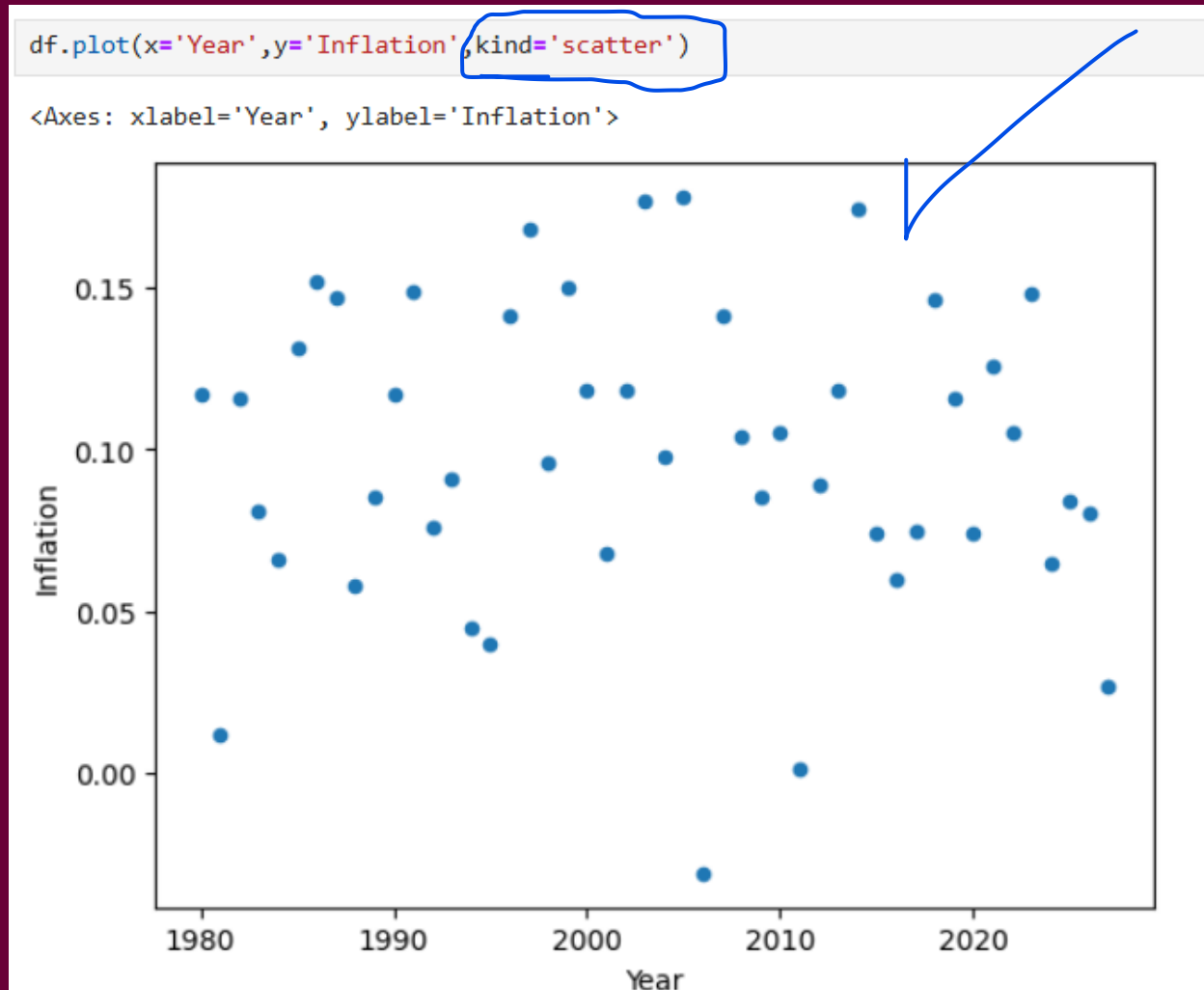


```
df.describe()
```

	Year	GDP_PPP	GDP_PerCapita_PPP	GDP_Nominal	GDP_PerCapita_Nominal	GDP_Growth	Inflation	Unemployment
count	48.00	48.000000	48.000000	48.000000	48.000000	48.000000	48.000000	48.000000
mean	2003.50	425.158333	4230.000000	236.200000	2350.000000	0.040917	0.099187	0.089625
std	14.00	229.264895	1983.843769	127.356234	1102.130092	0.018358	0.046383	0.019842
min	1980.00	40.300000	900.000000	22.400000	500.000000	0.006000	-0.031000	0.052000
25%	1991.75	232.725000	2564.975000	129.325000	1425.025000	0.030000	0.074000	0.074000
50%	2003.50	425.150000	4230.000000	236.200000	2350.000000	0.040000	0.101000	0.089500
75%	2015.25	617.600000	5895.025000	343.075000	3274.975000	0.051500	0.133500	0.098250
max	2027.00	810.000000	7560.000000	450.000000	4200.000000	0.082000	0.178000	0.139000

1. Scatter Plots

- A scatter plot is a type of chart used to display the relationship between two variables (variable) and a numerical variable (numerical).

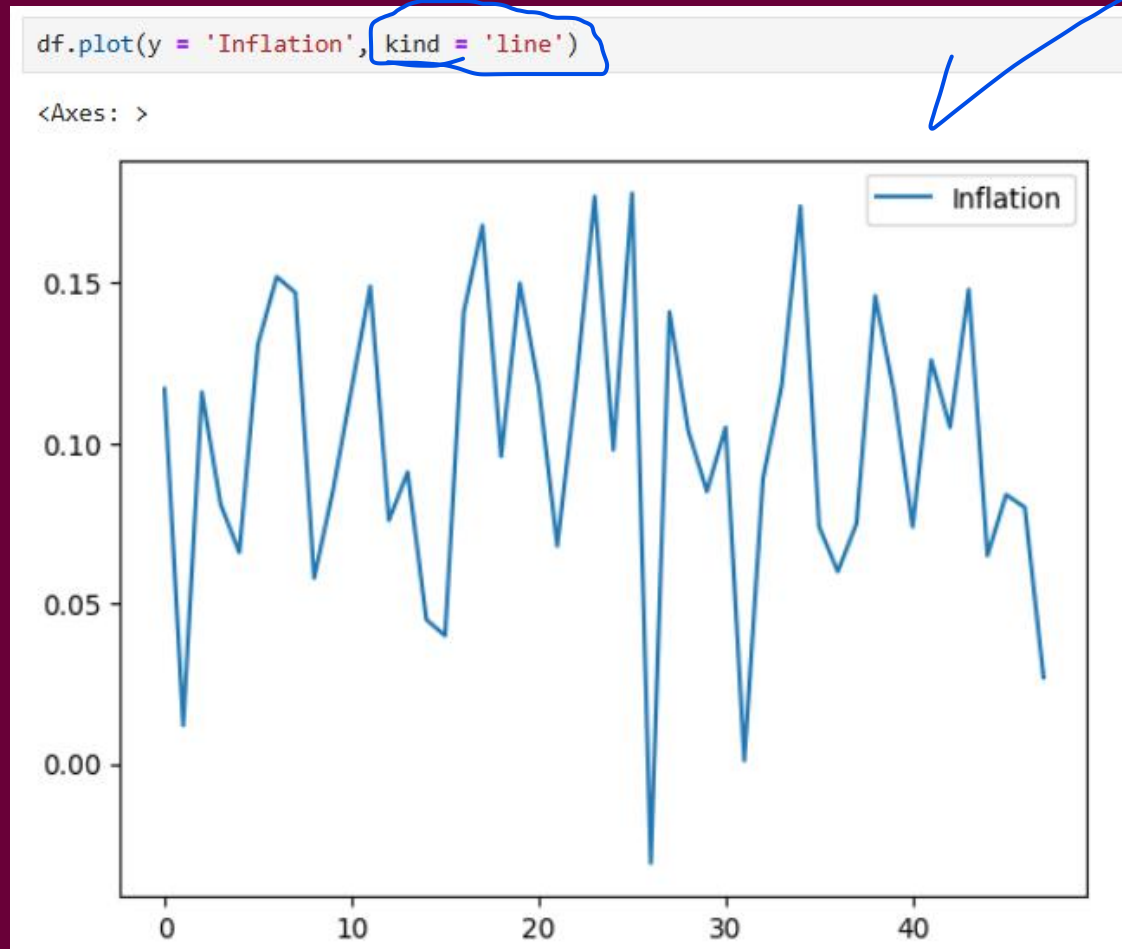


In your example, the chart shows how inflation changes over the years.

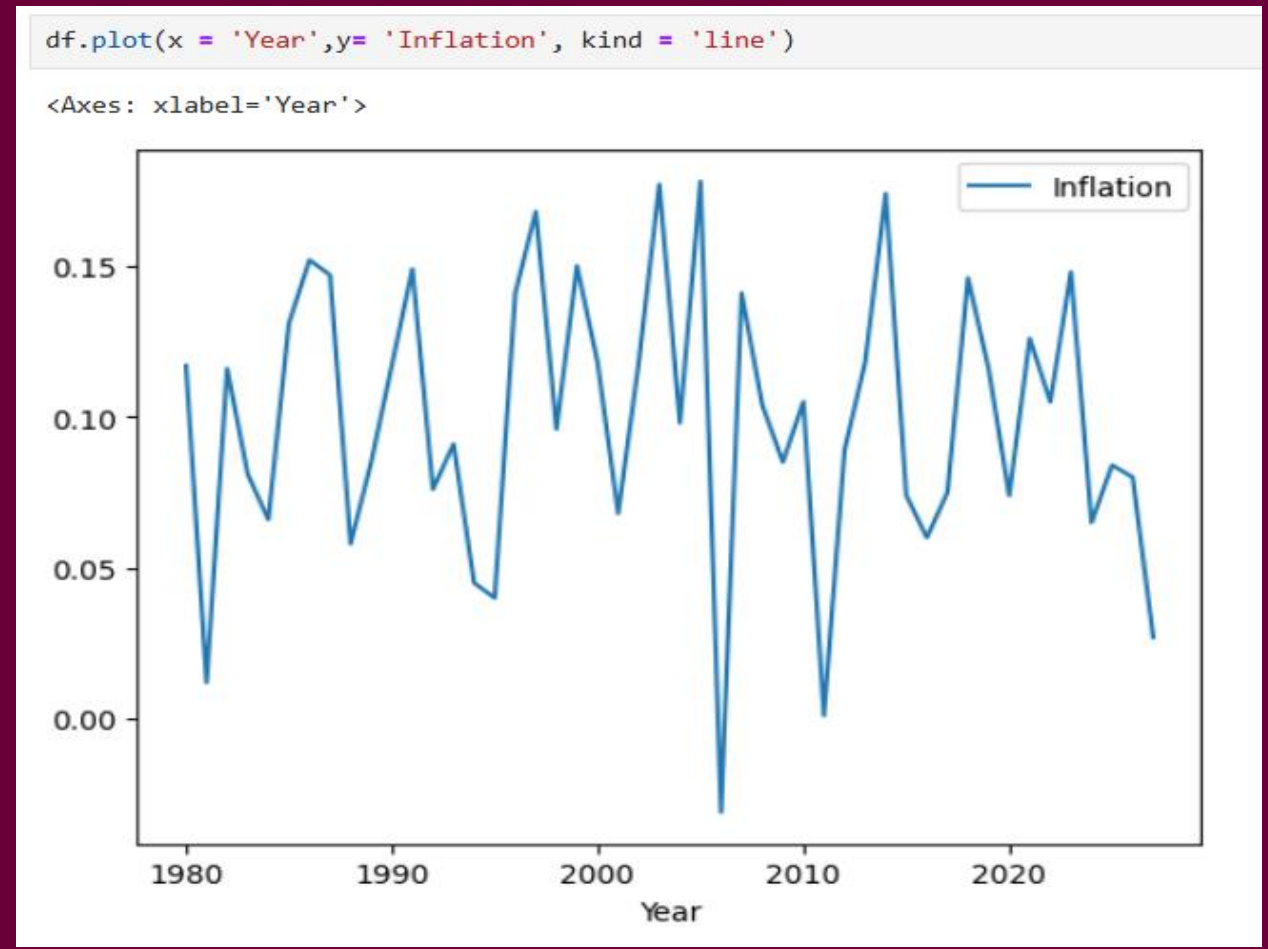
2. Line Plots

- A line plot is a type of chart used to display how data changes over time or in a specific order.
- It is very common in time series data such as inflation over the years, daily temperatures, and monthly sales.
- When do we use it? When we want to track how something changes over time. To see trends, seasonal patterns, or fluctuations.

There is no address for the X axis, because it uses the index.



Now The X-axis represents the years. The Y-axis represents inflation. The points are connected by a line to show the direction of inflation over the years.



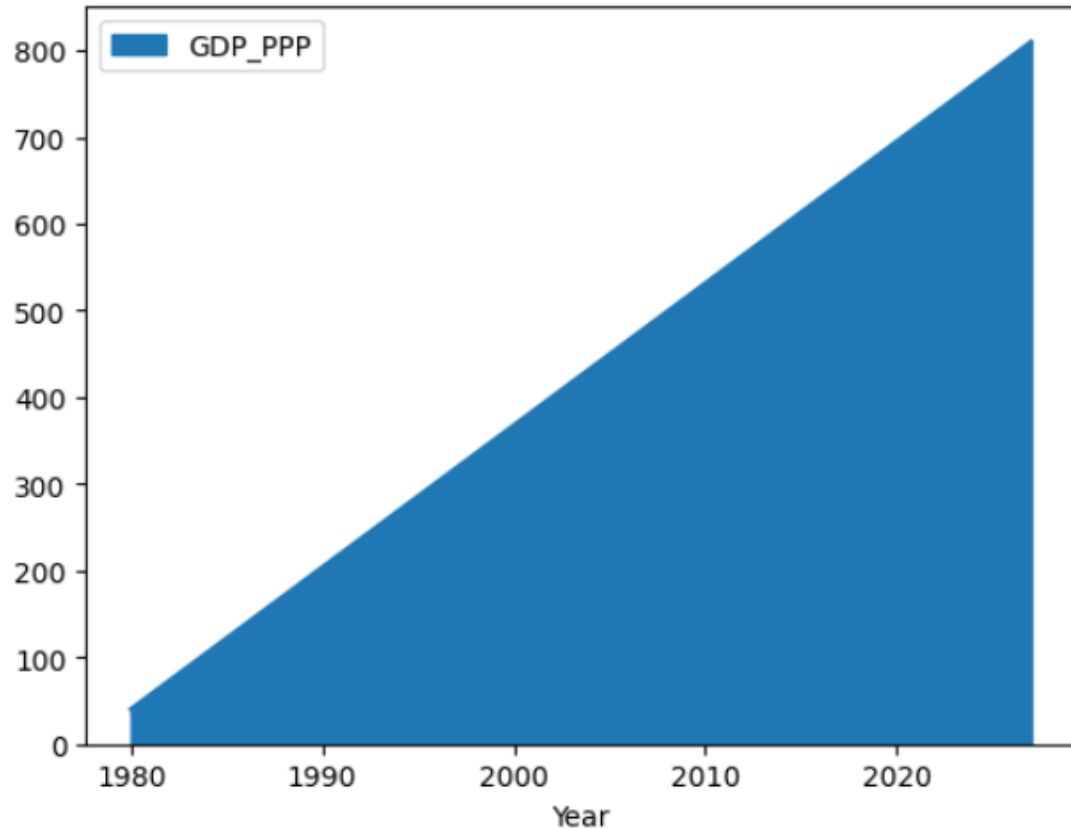
3. Area Plots

The X-axis represents years. The Y-axis represents GDP (Purchasing Power Parity).

- It is a type of chart like a line plot, but it highlights the area under the line with a specific color, giving a sense of the size or quantity covered by the variable.
- It is commonly used to compare values over time or to show the size of different contributions to a data set.
- It visually illustrates the accumulation or overall size of the data in a clear way.
- When do we use it? To track growth or decline over time. To compare the values of two or more variables. To make the chart clearer than a line plot when you want to emphasize the size.

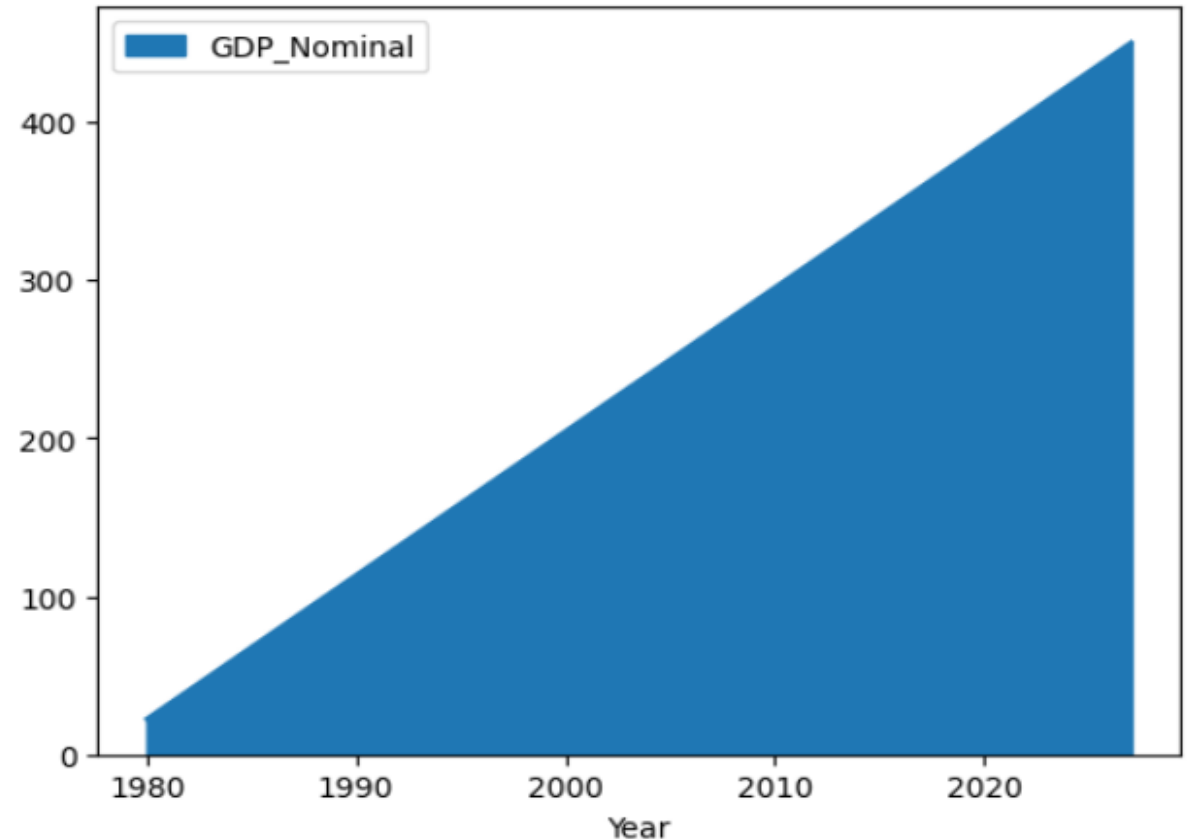
```
df.plot(x = 'Year', y = 'GDP_PPP', kind = 'area')
```

<Axes: xlabel='Year'>



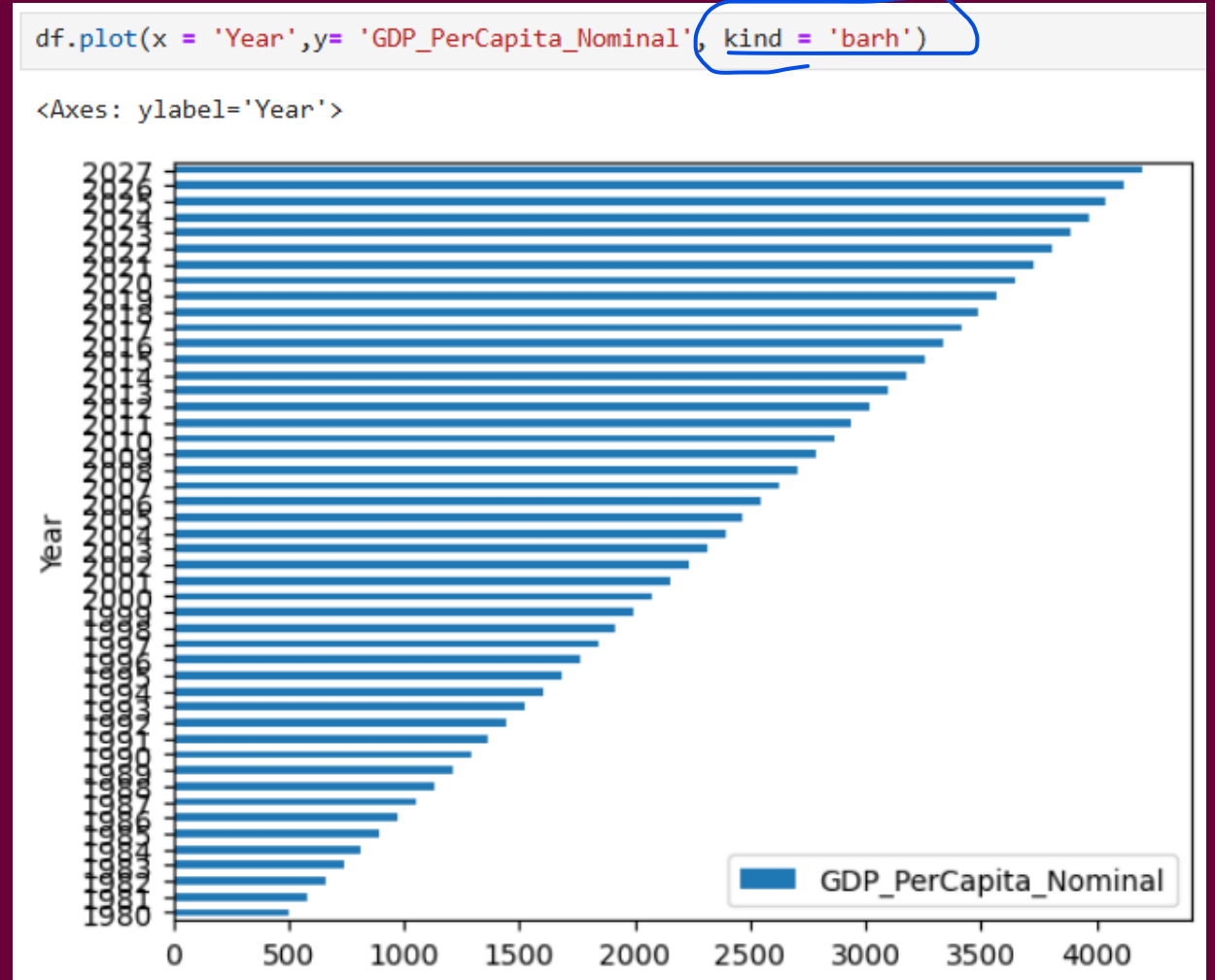
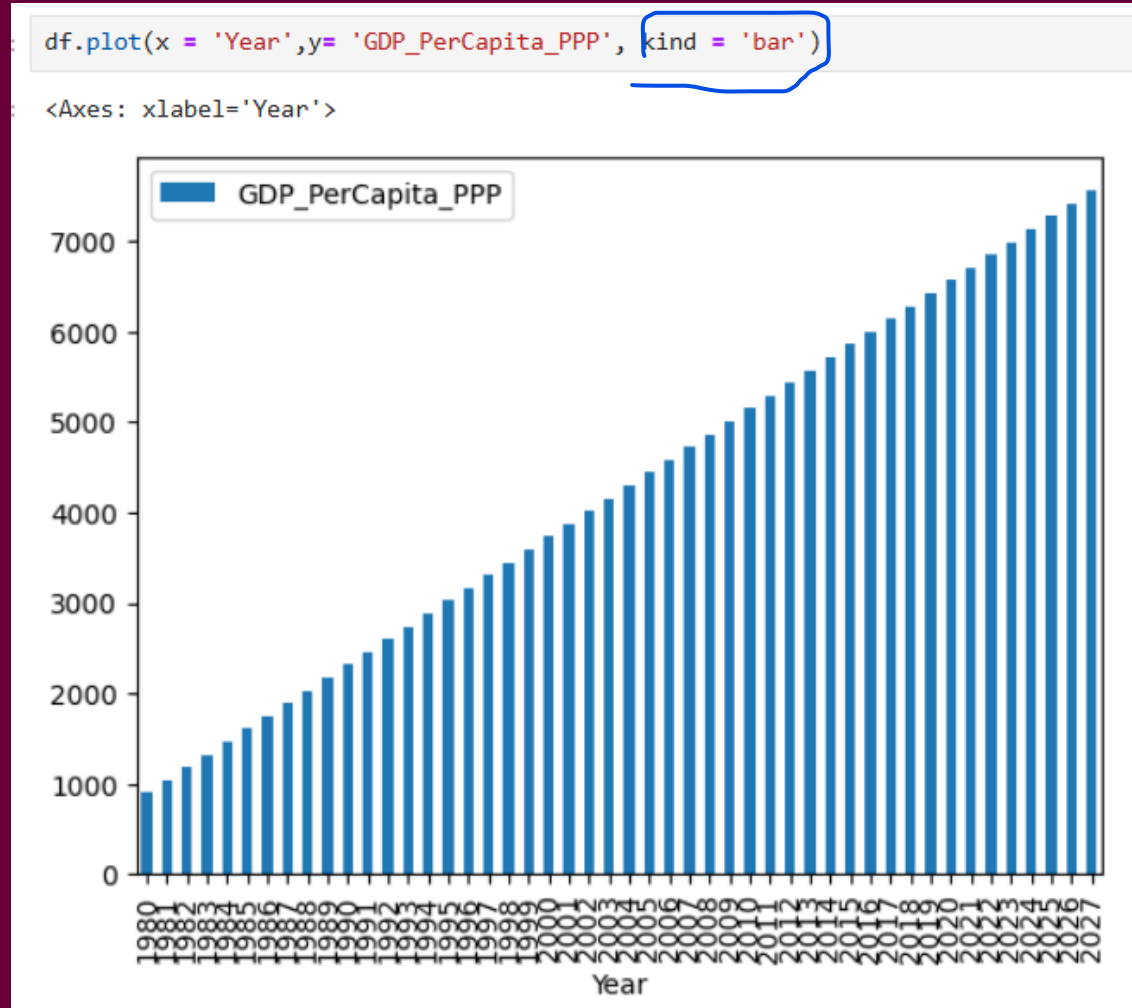
```
df.plot(x = 'Year', y = 'GDP_Nominal', kind = 'area')
```

<Axes: xlabel='Year'>



4. Bar Charts

- A bar chart is a type of chart used to visually compare different values using columns.
- Each column represents a value for a specific variable.
- Vertical Bar Plot: X-axis for categories, Y-axis for values.
- Horizontal Bar Plot: Y-axis for categories, X-axis for values.
- When do we use it? To compare different values simultaneously. To see which categories are larger or smaller. Good for non-time data or for categories such as countries, years, and products.



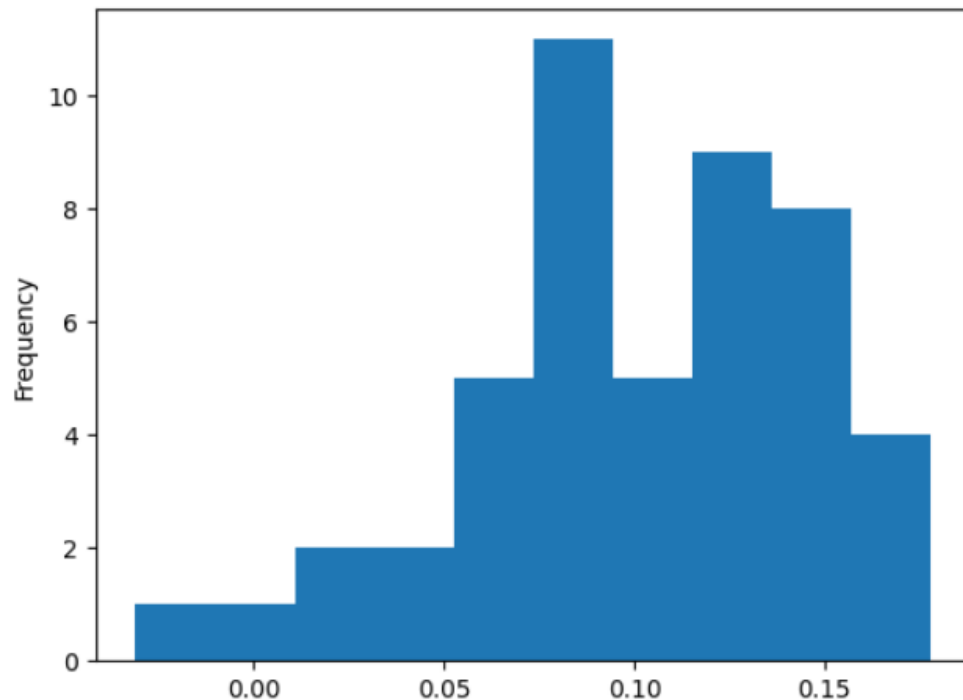
5. Histograms

- A histogram is a type of chart used to represent the distribution of numerical data. It displays the number of values (frequency) that fall within specific categories (bins). A bin is a small range of values, such as 0–5, 5–10, and so on. Each column represents the number of values within that range.
- It's useful for understanding data distribution: Is it normally distributed? Does it have skewness? Does it have outliers?
- When do we use it? To determine the distribution of a single variable. To check if data is clustered or sparse. To easily compare the distributions of different data sets.

We plot a histogram for the inflation variable. Pandas automatically divides the values into several default bins (usually 10).

```
df['Inflation'].plot(kind = 'hist')
```

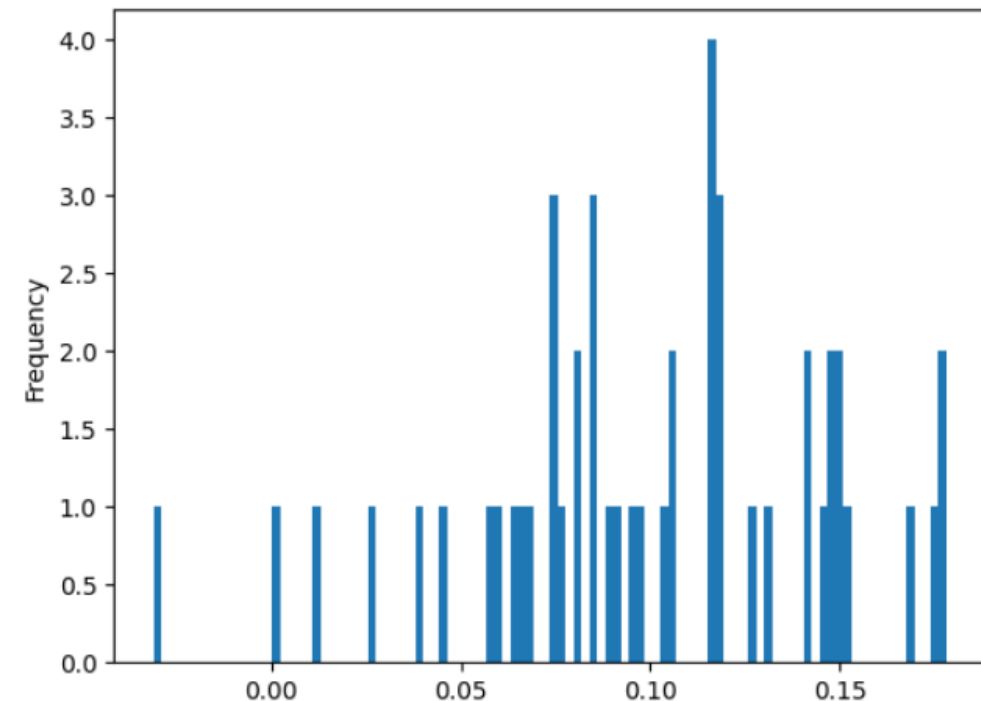
<Axes: ylabel='Frequency'>



The same variable, but the number of bins = 100. This provides a more detailed breakdown of the data distribution.

```
df['Inflation'].plot(kind = 'hist', bins = 100)
```

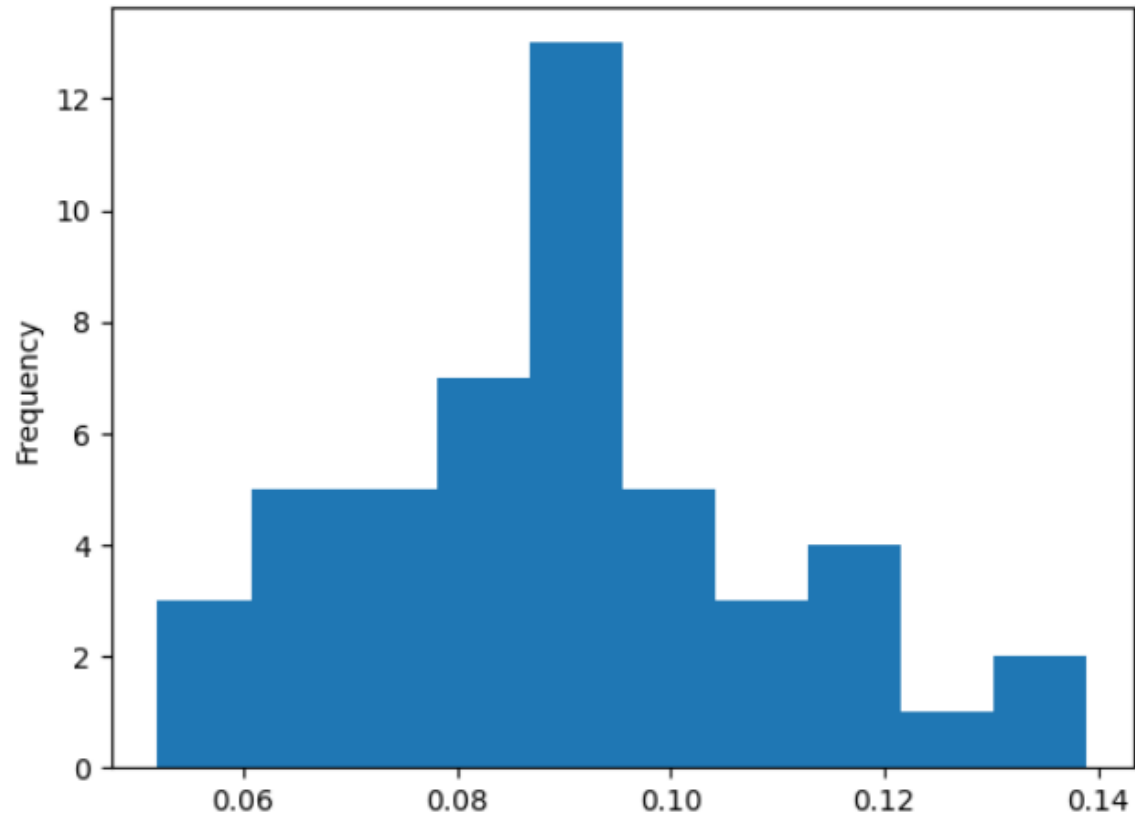
<Axes: ylabel='Frequency'>



The distribution of unemployment is shown.

```
df['Unemployment'].plot(kind = 'hist')
```

<Axes: ylabel='Frequency'>



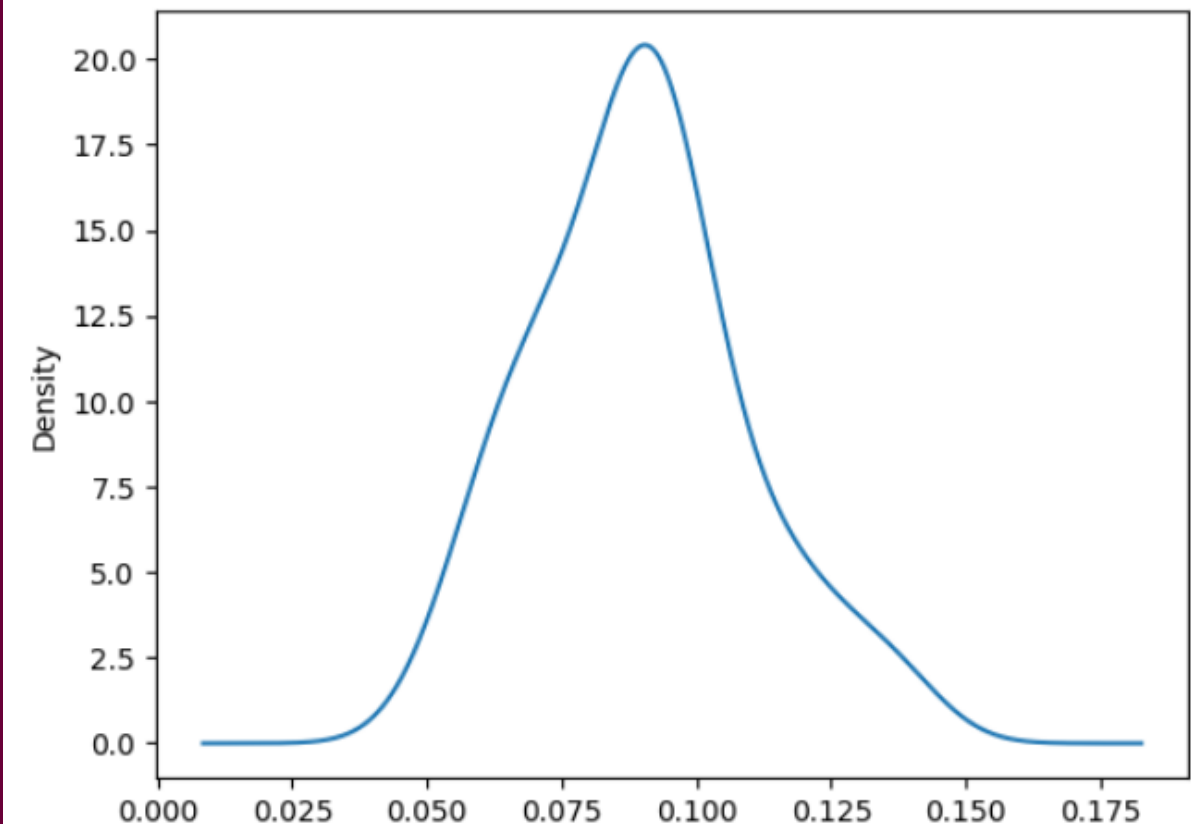
KDE = Kernel Density Estimate. The data distribution is shown as a smooth line instead of bars.

ylabel='Density' → The Y-axis represents density, not numbers.

The unemployment distribution is shown more smoothly than as bars.

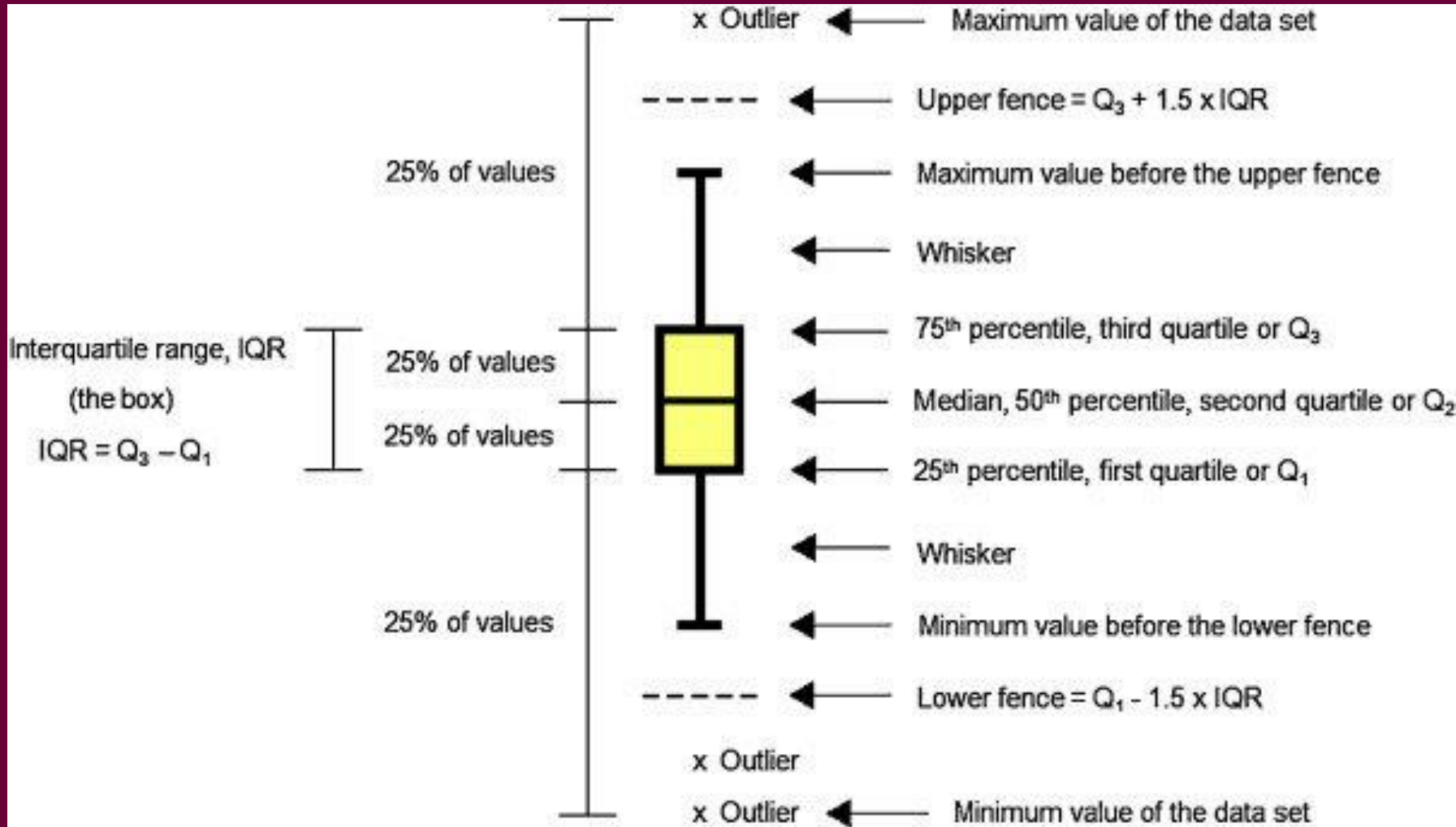
```
df['Unemployment'].plot(kind = 'kde')
```

<Axes: ylabel='Density'>



6. Box Plot

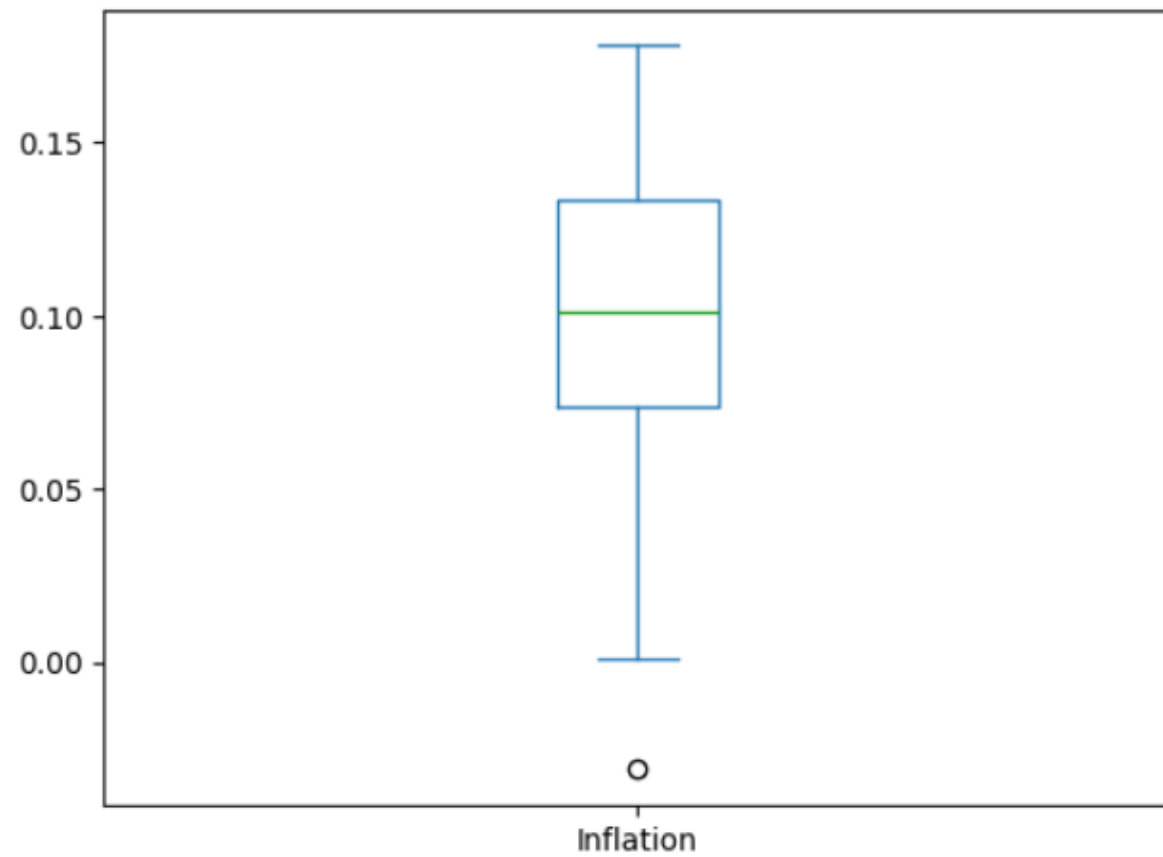
- A box plot (or box and whisker chart) is a powerful visual tool that summarizes the distribution of numerical data and reveals outliers.
- Why Use a Box Plot? To review the dispersion and distribution of data. To determine the median, minimum, maximum, and interquartile range. To quickly identify outliers.



- Components of a Box Plot:
- The Box: Represents the first quartile (Q_1) to the third quartile (Q_3).
- Inside the box, the center line often represents the median.
- The Whiskers: Lines extending from the box represent the maximum and minimum acceptable values (typically $\pm 1.5 \times IQR$).
- Individual points outside the whiskers: Represent outliers.

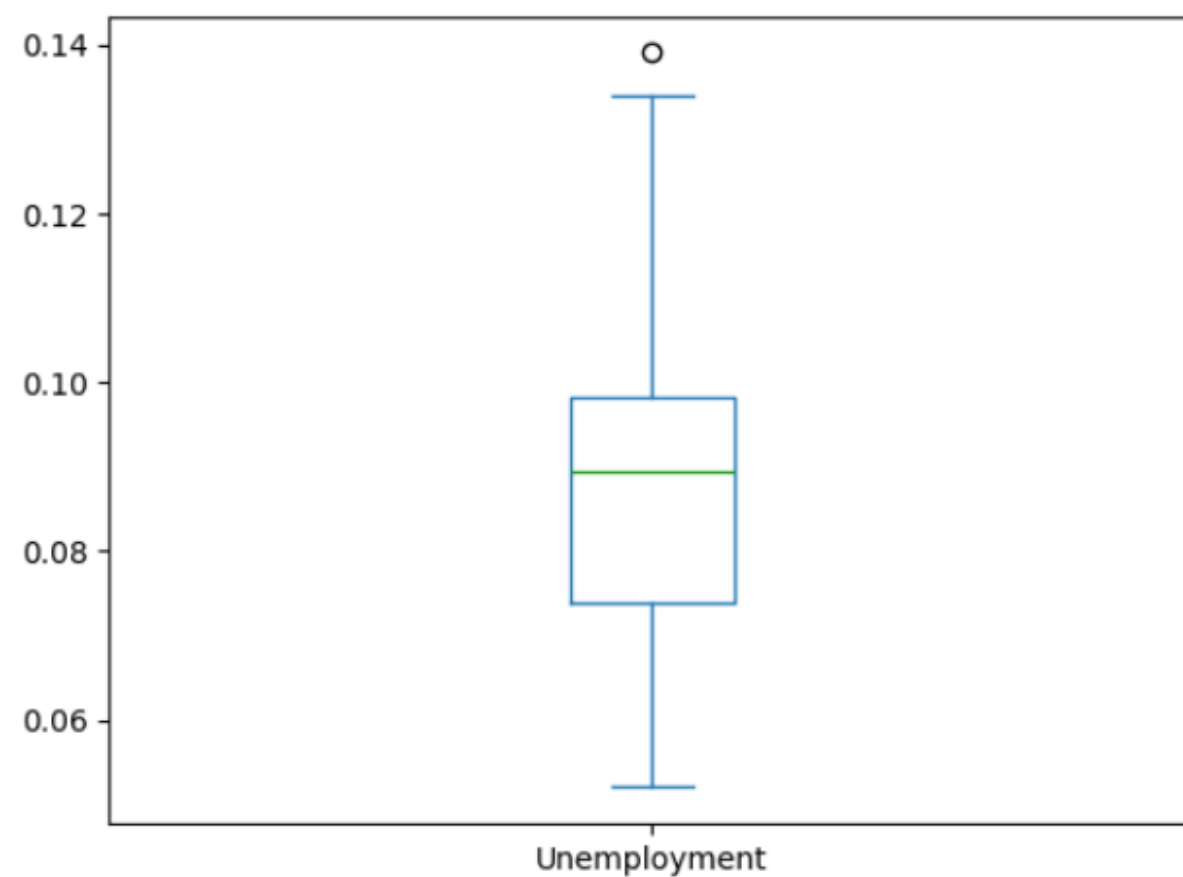
```
df['Inflation'].plot(kind = 'box')
```

<Axes: >



```
df['Unemployment'].plot(kind = 'box')
```

<Axes: >



7. Pie Charts

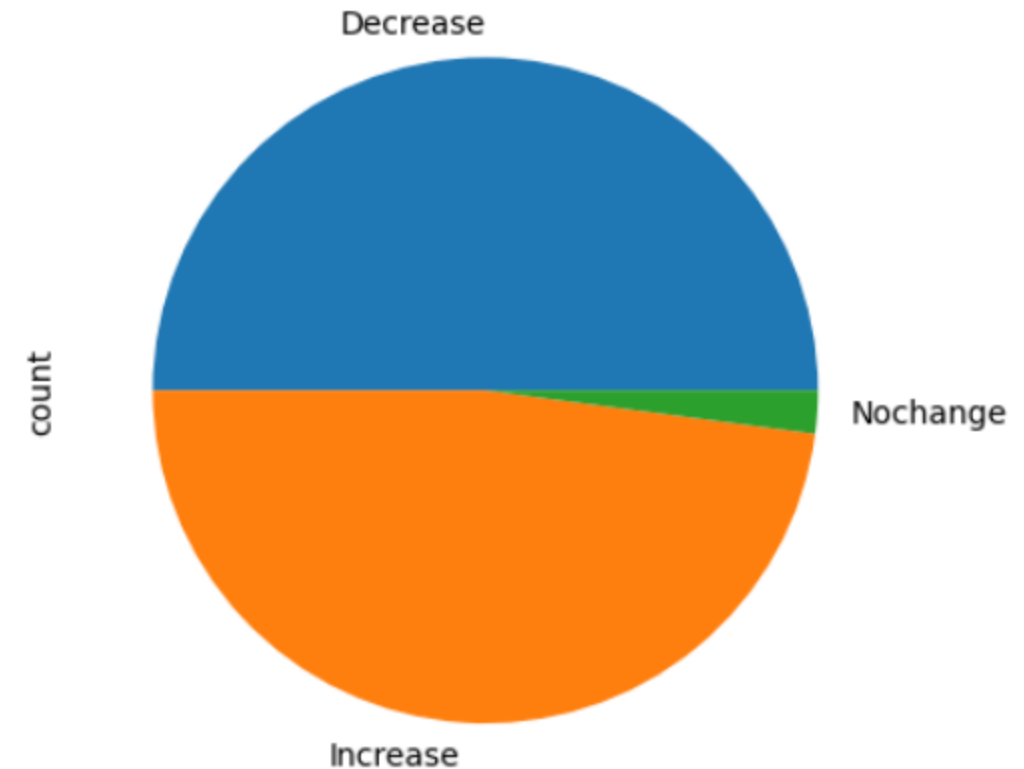
- A pie chart is a type of chart used to illustrate the percentages of parts of a whole.
- Each segment of the pie chart represents a specific category of data.
- The size of each segment reflects the size of that category relative to the other categories.
- It's very useful for quickly understanding the distribution of percentages among different categories visually.
- When do we use it? When we want to know the percentage of each category in the total. When working with categorical data.

```
df['Inflation_Change'].value_counts()
```

```
Inflation_Change  
Decrease      24  
Increase      23  
Nochange       1  
Name: count, dtype: int64
```

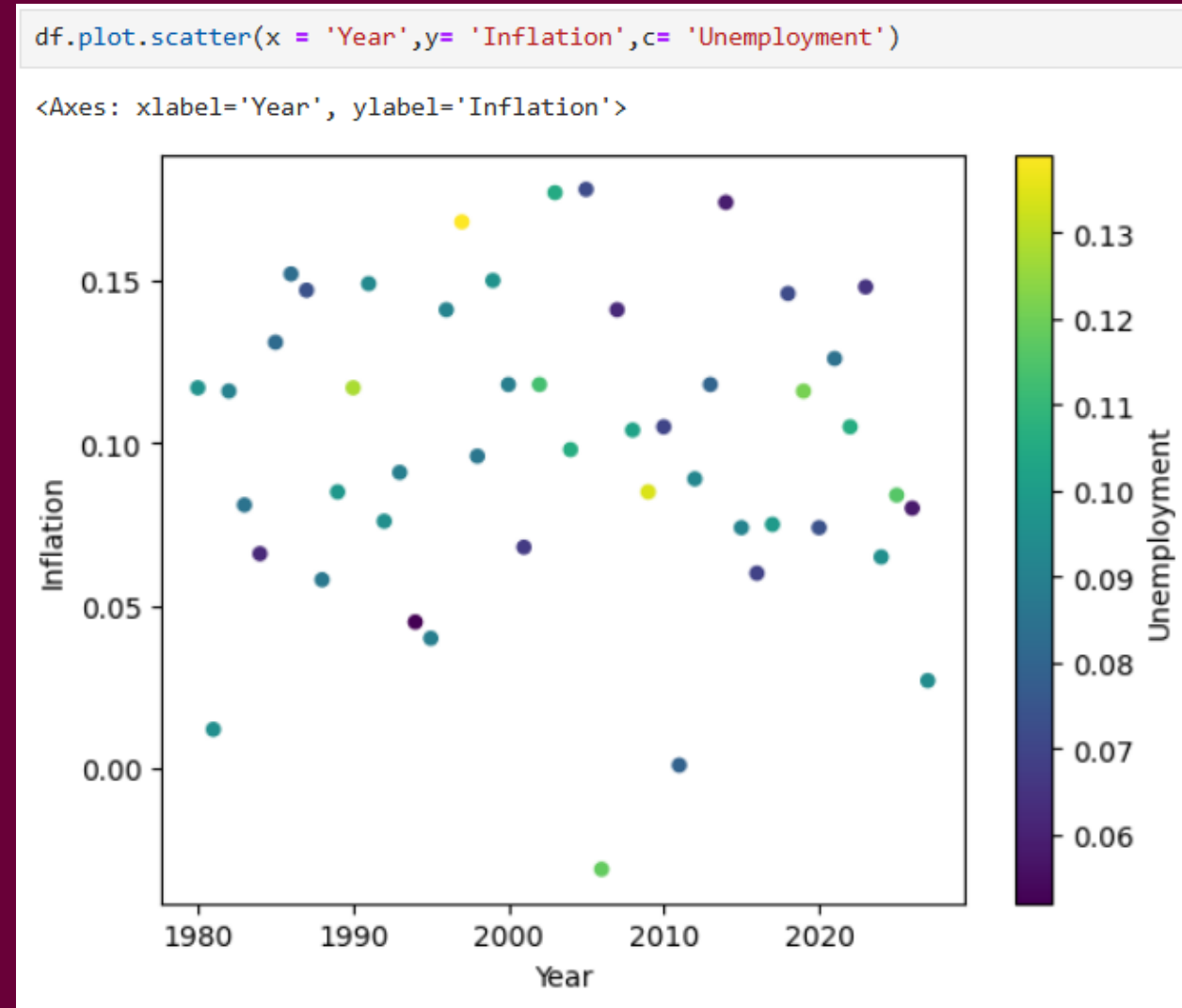
```
df['Inflation_Change'].value_counts().plot(kind = 'pie')
```

<Axes: ylabel='count'>



8. Color Map (Colored Scatter Plot)

- A color map here means we use colors to add a third dimension to the data on a scatter plot.
- X-axis → represents an independent variable (e.g., Year).
- Y-axis → represents a dependent variable (e.g., Inflation).
- The color of each point → represents the value of a third variable (e.g., Unemployment).
- In this way, we can see a three-dimensional relationship in a two-dimensional way, using color as the third axis.
- When do we use it? When we want to see the effect of a third variable on the relationship between two variables. To analyze patterns and clusters within the data.



Data Preprocessing

- Data Preprocessing is the process of cleaning, transforming, and preparing data for analysis. It is a crucial step in the Data Mining pipeline as it ensures that the data is in a format that can be easily and accurately analyzed.



1. DEALING WITH MISSING VALUES



Missing values are commonly found in data, and we need to handle them to ensure the accuracy and reliability of the results.



Objectives: Remove or replace missing values appropriately. Use different methods depending on the data type and the importance of the missing values.

How to discover missing values?

```
df_small.isnull()
```

	Year	GDP_PPP	GDP_PerCapita_PPP	GDP_Nominal	GDP_PerCapita_Nominal	GDP_Growth	Inflation	Unemployment	Inflation_Change
0	True	False	False	False	False	False	False	False	False
1	False	False	False	True	False	False	False	False	False
2	False	False	False	False	False	True	False	False	False
3	False	False	False	True	False	False	False	False	False
4	False	False	False	False	False	True	False	False	False
5	False	False	False	True	False	False	False	False	False
6	False	False	False	False	False	True	False	False	False
7	False	False	False	True	False	False	False	False	False
8	False	False	False	False	False	True	False	False	False
9	False	False	False	False	False	True	False	False	False
10	False	False	False	False	False	True	False	False	False
11	False	False	False	False	False	True	False	False	False
12	False	False	False	False	False	False	False	False	False

isnull() ✓

isnull.sum() ✓

```
df_small.isnull().sum()
```

```
Year          1
GDP_PPP        0
GDP_PerCapita_PPP  0
GDP_Nominal    4
GDP_PerCapita_Nominal  0
GDP_Growth     7
Inflation      0
Unemployment   0
Inflation_Change  0
dtype: int64
```



Handling Missing Values

Delete rows containing any missing information.

```
df2 = df_small.dropna()  
df2
```

	Year	GDP_PPP	GDP_PerCapita_PPP	GDP_Nominal	GDP_PerCapita_Nominal	GDP_Growth	Inflation	Unemployment	Inflation_Change
12	1992.0	236.8	2600.4	131.6	1444.7	0.05	0.076	0.095	Decrease

1.1 Dropping Missing Values

Row 12

Delete columns containing any missing information.

```
df2 = df_small.dropna(axis=1)  
df2
```

	GDP_PPP	GDP_PerCapita_PPP	GDP_PerCapita_Nominal	Inflation	Unemployment	Inflation_Change
0	40.3	900.0	500.0	0.117	0.096	Nochange
1	56.7	1041.7	578.7	0.012	0.095	Decrease
2	73.1	1183.4	657.4	0.116	0.090	Increase
3	89.4	1325.1	736.2	0.081	0.085	Decrease
4	105.8	1466.8	814.9	0.066	0.062	Decrease
5	122.2	1608.5	893.6	0.131	0.082	Increase
6	138.6	1750.2	972.3	0.152	0.083	Increase
7	155.0	1891.9	1051.1	0.147	0.074	Decrease
8	171.3	2033.6	1129.8	0.058	0.087	Decrease
9	187.7	2175.3	1208.5	0.085	0.098	Increase
10	204.1	2317.0	1287.2	0.117	0.128	Increase
11	220.5	2458.7	1366.0	0.149	0.093	Increase
12	236.8	2600.4	1444.7	0.076	0.095	Decrease

Using `thresh` on rows: `thresh=n` Keep if there are at least n integer values

```
df2 = df_small.dropna(thresh=3)  
df2
```

	Year	GDP_PPP	GDP_PerCapita_PPP	GDP_Nominal	GDP_PerCapita_Nominal	GDP_Growth	Inflation	Unemployment	Inflation_Change
0	NaN	40.3	900.0	22.4	500.0	0.055	0.117	0.096	Nochange
1	1981.0	56.7	1041.7	NaN	578.7	0.042	0.012	0.095	Decrease
2	1982.0	73.1	1183.4	40.6	657.4	NaN	0.116	0.090	Increase
3	1983.0	89.4	1325.1	NaN	736.2	0.075	0.081	0.085	Decrease
4	1984.0	105.8	1466.8	58.8	814.9	NaN	0.066	0.062	Decrease
5	1985.0	122.2	1608.5	NaN	893.6	0.040	0.131	0.082	Increase
6	1986.0	138.6	1750.2	77.0	972.3	NaN	0.152	0.083	Increase
7	1987.0	155.0	1891.9	NaN	1051.1	0.060	0.147	0.074	Decrease
8	1988.0	171.3	2033.6	95.2	1129.8	NaN	0.058	0.087	Decrease
9	1989.0	187.7	2175.3	104.3	1208.5	NaN	0.085	0.098	Increase
10	1990.0	204.1	2317.0	113.4	1287.2	NaN	0.117	0.128	Increase
11	1991.0	220.5	2458.7	122.5	1366.0	NaN	0.149	0.093	Increase
12	1992.0	236.8	2600.4	131.6	1444.7	0.050	0.076	0.095	Decrease

Keep the row
if it has at
least 3 correct
values.

Using `thresh` on columns:

`thresh = n` Keep at least n integer values.

```
df2 = df_small.dropna(axis=1, thresh=7)
df2
```

	Year	GDP_PPP	GDP_PerCapita_PPP	GDP_Nominal	GDP_PerCapita_Nominal	Inflation	Unemployment	Inflation_Change
0	NaN	40.3	900.0	22.4	500.0	0.117	0.096	Nochange
1	1981.0	56.7	1041.7	NaN	578.7	0.012	0.095	Decrease
2	1982.0	73.1	1183.4	40.6	657.4	0.116	0.090	Increase
3	1983.0	89.4	1325.1	NaN	736.2	0.081	0.085	Decrease
4	1984.0	105.8	1466.8	58.8	814.9	0.066	0.062	Decrease
5	1985.0	122.2	1608.5	NaN	893.6	0.131	0.082	Increase
6	1986.0	138.6	1750.2	77.0	972.3	0.152	0.083	Increase
7	1987.0	155.0	1891.9	NaN	1051.1	0.147	0.074	Decrease
8	1988.0	171.3	2033.6	95.2	1129.8	0.058	0.087	Decrease
9	1989.0	187.7	2175.3	104.3	1208.5	0.085	0.098	Increase
10	1990.0	204.1	2317.0	113.4	1287.2	0.117	0.128	Increase
11	1991.0	220.5	2458.7	122.5	1366.0	0.149	0.093	Increase
12	1992.0	236.8	2600.4	131.6	1444.7	0.076	0.095	Decrease

Keep the column
if it contains at
least 7 valid values

1.2 Filling with Constant

Replace all missing values with a single fixed value

```
df2 = df_small.fillna('NA')
df2
```

	Year	GDP_PPP	GDP_PerCapita_PPP	GDP_Nominal	GDP_PerCapita_Nominal	GDP_Growth	Inflation	Unemployment	Inflation_Change
0	NA	40.3	900.0	22.4	500.0	0.055	0.117	0.096	Nochange
1	1981.0	56.7	1041.7	NA	578.7	0.042	0.012	0.095	Decrease
2	1982.0	73.1	1183.4	40.6	657.4	NA	0.116	0.090	Increase
3	1983.0	89.4	1325.1	NA	736.2	0.075	0.081	0.085	Decrease
4	1984.0	105.8	1466.8	58.8	814.9	NA	0.066	0.062	Decrease
5	1985.0	122.2	1608.5	NA	893.6	0.04	0.131	0.082	Increase
6	1986.0	138.6	1750.2	77.0	972.3	NA	0.152	0.083	Increase
7	1987.0	155.0	1891.9	NA	1051.1	0.06	0.147	0.074	Decrease
8	1988.0	171.3	2033.6	95.2	1129.8	NA	0.058	0.087	Decrease
9	1989.0	187.7	2175.3	104.3	1208.5	NA	0.085	0.098	Increase
10	1990.0	204.1	2317.0	113.4	1287.2	NA	0.117	0.128	Increase
11	1991.0	220.5	2458.7	122.5	1366.0	NA	0.149	0.093	Increase
12	1992.0	236.8	2600.4	131.6	1444.7	0.05	0.076	0.095	Decrease

Any NaN in any column will be converted to "NA".

When you fill numeric columns with the text value "NA", the column type will change to object instead of float. This can cause problems in calculations later, so we must consider the data type.

```
df2.dtypes
```

```
Year          object
GDP_PPP       float64
GDP_PerCapita_PPP float64
GDP_Nominal    object
GDP_PerCapita_Nominal float64
GDP_Growth     object
Inflation      float64
Unemployment   float64
Inflation_Change object
dtype: object
```

Replace only a specific column with a fixed value

We created a new column called `Year_filled`.
If there is a NaN in Year, it will be changed to "YEAR".
The remaining values will remain as they are.

```
df_small['Year_filled'] = df_small['Year'].fillna('YEAR')  
df_small
```

	Year	GDP_PPP	GDP_PerCapita_PPP	GDP_Nominal	GDP_PerCapita_Nominal	GDP_Growth	Inflation	Unemployment	Inflation_Change	Year_filled
0	NaN	40.3	900.0	22.4	500.0	0.055	0.117	0.096	Nochange	YEAR
1	1981.0	56.7	1041.7	NaN	578.7	0.042	0.012	0.095	Decrease	1981.0
2	1982.0	73.1	1183.4	40.6	657.4	NaN	0.116	0.090	Increase	1982.0
3	1983.0	89.4	1325.1	NaN	736.2	0.075	0.081	0.085	Decrease	1983.0
4	1984.0	105.8	1466.8	58.8	814.9	NaN	0.066	0.062	Decrease	1984.0
5	1985.0	122.2	1608.5	NaN	893.6	0.040	0.131	0.082	Increase	1985.0
6	1986.0	138.6	1750.2	77.0	972.3	NaN	0.152	0.083	Increase	1986.0
7	1987.0	155.0	1891.9	NaN	1051.1	0.060	0.147	0.074	Decrease	1987.0
8	1988.0	171.3	2033.6	95.2	1129.8	NaN	0.058	0.087	Decrease	1988.0
9	1989.0	187.7	2175.3	104.3	1208.5	NaN	0.085	0.098	Increase	1989.0
10	1990.0	204.1	2317.0	113.4	1287.2	NaN	0.117	0.128	Increase	1990.0
11	1991.0	220.5	2458.7	122.5	1366.0	NaN	0.149	0.093	Increase	1991.0
12	1992.0	236.8	2600.4	131.6	1444.7	0.050	0.076	0.095	Decrease	1992.0

If the column is numerical, it's best to fill it in with a number, not text:

```
df_small['GDP_Growth_filled'] = df_small['GDP_Growth'].fillna(0)  
df_small
```

	Year	GDP_PPP	GDP_PerCapita_PPP	GDP_Nominal	GDP_PerCapita_Nominal	GDP_Growth	Inflation	Unemployment	Inflation_Change	Year_filled	GDP_Growth_filled
1	NaN	40.3	900.0	22.4	500.0	0.055	0.117	0.096	Nochange	YEAR	0.055
2	1981.0	56.7	1041.7	NaN	578.7	0.042	0.012	0.095	Decrease	1981.0	0.042
3	1982.0	73.1	1183.4	40.6	657.4	NaN	0.116	0.090	Increase	1982.0	0.000
4	1983.0	89.4	1325.1	NaN	736.2	0.075	0.081	0.085	Decrease	1983.0	0.075
5	1984.0	105.8	1466.8	58.8	814.9	NaN	0.066	0.062	Decrease	1984.0	0.000
6	1985.0	122.2	1608.5	NaN	893.6	0.040	0.131	0.082	Increase	1985.0	0.040
7	1986.0	138.6	1750.2	77.0	972.3	NaN	0.152	0.083	Increase	1986.0	0.000
8	1987.0	155.0	1891.9	NaN	1051.1	0.060	0.147	0.074	Decrease	1987.0	0.060
9	1988.0	171.3	2033.6	95.2	1129.8	NaN	0.058	0.087	Decrease	1988.0	0.000
10	1989.0	187.7	2175.3	104.3	1208.5	NaN	0.085	0.098	Increase	1989.0	0.000
11	1990.0	204.1	2317.0	113.4	1287.2	NaN	0.117	0.128	Increase	1990.0	0.000
12	1991.0	220.5	2458.7	122.5	1366.0	NaN	0.149	0.093	Increase	1991.0	0.000
13	1992.0	236.8	2600.4	131.6	1444.7	0.050	0.076	0.095	Decrease	1992.0	0.050

1.3 Filling with ffill

- It takes the last integer value before it and puts it in place of the NaN

```
df_small['GDP_filled_ffill'] = df_small['GDP_Nominal'].fillna(method='ffill')  
df_small[['GDP_Nominal', 'GDP_filled_ffill']]
```

	GDP_Nominal	GDP_filled_ffill
0	22.4	22.4
1	NaN	22.4
2	40.6	40.6
3	NaN	40.6
4	58.8	58.8
5	NaN	58.8
6	77.0	77.0
7	NaN	77.0
8	95.2	95.2
9	104.3	104.3
10	113.4	113.4
11	122.5	122.5
12	131.6	131.6

1.4 Filling with bfill

- It takes the first correct value after it and puts it in place of the NaN.

```
df_small['GDP_filled_bfill'] = df_small['GDP_Nominal'].fillna(method='bfill')  
df_small[['GDP_Nominal', 'GDP_filled_bfill']]
```

	GDP_Nominal	GDP_filled_bfill
0	22.4	22.4
1	NaN	40.6
2	40.6	40.6
3	NaN	58.8
4	58.8	58.8
5	NaN	77.0
6	77.0	77.0
7	NaN	95.2
8	95.2	95.2
9	104.3	104.3
10	113.4	113.4
11	122.5	122.5
12	131.6	131.6

1.5 Filling with mean

```
df_small['GDP_Nominal_filled_mean'] = df_small['GDP_Nominal'].fillna(  
    df_small['GDP_Nominal'].mean()  
)  
  
df_small[['GDP_Nominal', 'GDP_Nominal_filled_mean']]
```

	GDP_Nominal	GDP_Nominal_filled_mean
0	22.4	22.400000
1	NaN	85.088889
2	40.6	40.600000
3	NaN	85.088889
4	58.8	58.800000
5	NaN	85.088889
6	77.0	77.000000
7	NaN	85.088889
8	95.2	95.200000
9	104.3	104.300000
10	113.4	113.400000
11	122.5	122.500000
12	131.6	131.600000

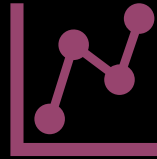
1.6 Filling with Mode

```
df_small['GDP_Nominal_filled_mode'] = df_small['GDP_Nominal'].fillna(  
    df_small['GDP_Nominal'].mode()[0]  
)
```

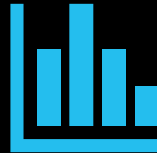
```
df_small[['GDP_Nominal', 'GDP_Nominal_filled_mode']]
```

	GDP_Nominal	GDP_Nominal_filled_mode
0	22.4	22.4
1	NaN	22.4
2	40.6	40.6
3	NaN	22.4
4	58.8	58.8
5	NaN	22.4
6	77.0	77.0
7	NaN	22.4
8	95.2	95.2
9	104.3	104.3
10	113.4	113.4
11	122.5	122.5
12	131.6	131.6

2. DEALING WITH OUTLIERS



In the world of data analysis, outliers are data points that deviate significantly from the typical distribution of a dataset.



Detecting outliers is a crucial step in data preprocessing and analysis, as these unusual data points can distort statistical measures and lead to inaccurate insights.

2.1 Detect Outliers Using IQR

- IQR is $Q3 - Q1$, and we can set the lower and upper bound by $Q1 - 1.5IQR$ and $Q3 + 1.5IQR$.
- Box plot automatically draws the lower/upper bound for us. We can also detect the data by defining a function.



	high	low
0	31	21
1	44	22
2	39	12
3	35	24
4	32	16
5	60	10
6	43	13
7	35	22
8	35	13
9	28	23

```
df3.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 118 entries, 0 to 117
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0   high    118 non-null    int32  
 1   low     118 non-null    int32  
dtypes: int32(2)
memory usage: 1.1 KB
```

```
df3.shape
```

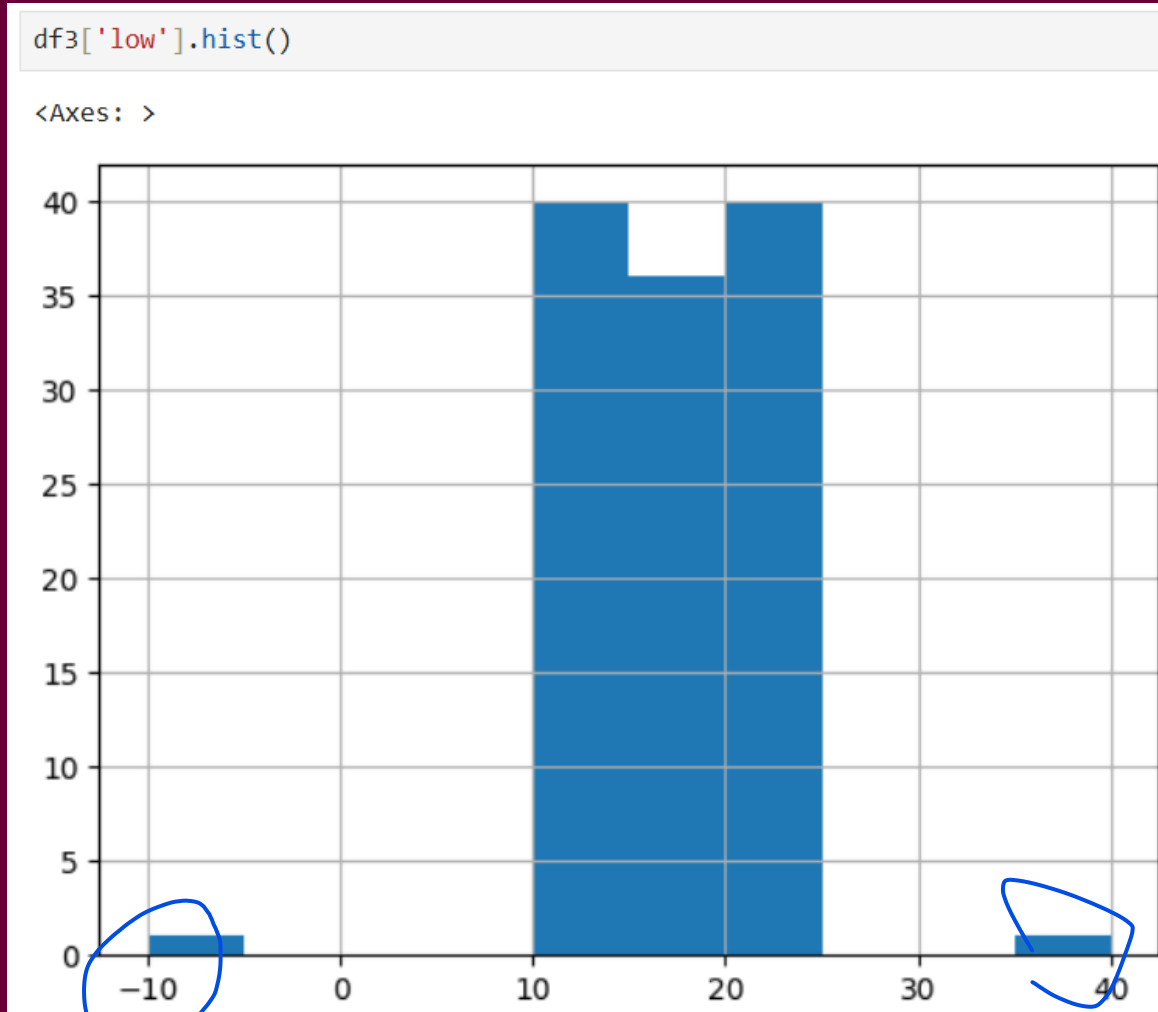
```
(118, 2)
```

```
df3.describe()
```

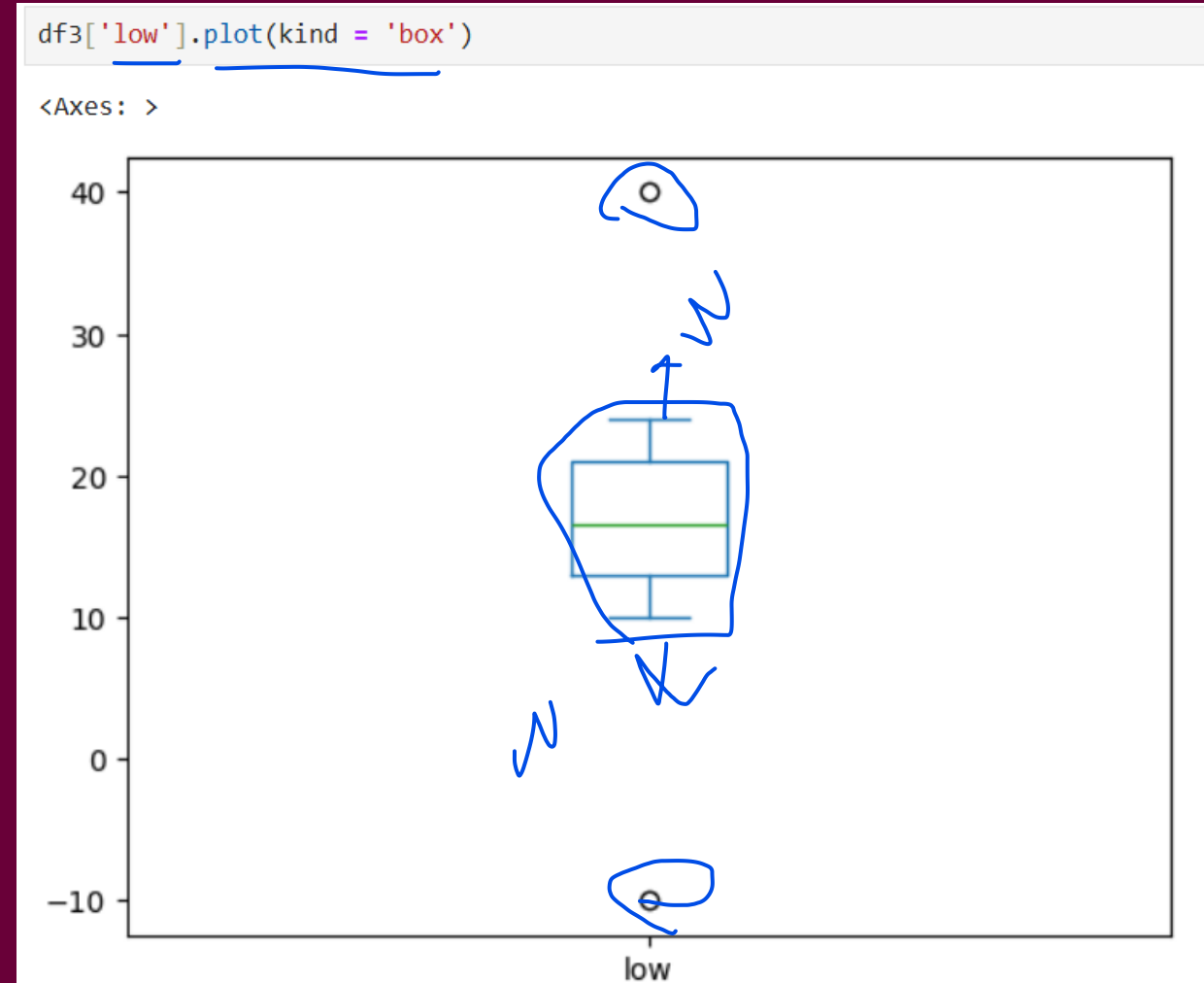
	high	low
count	118.000000	118.000000
mean	33.711864	16.754237
std	6.576443	5.445827
min	5.000000	-10.000000
25%	29.000000	13.000000
50%	33.000000	16.500000
75%	39.000000	21.000000
max	60.000000	40.000000

Check for Outliers in df['low']

The Distribution Plot before Removing Outliers



The Box Plot before Removing Outliers



Setup Thresholds

```
# Calculate IQR
low_IQR = df3['low'].quantile(0.75) - df3['low'].quantile(0.25)

# Lower Fence , Upper Fence
low_low_limit = df3['low'].quantile(0.25) - 1.5 * low_IQR
low_high_limit = df3['low'].quantile(0.75) + 1.5 * low_IQR

print("Lower Limit:", low_low_limit)
print("Upper Limit:", low_high_limit)
```

```
Lower Limit: 1.0
Upper Limit: 33.0
```

- Here, the $1.5 \times \text{IQR}$ rule is used to identify outliers.
- Any value less than 1.0 or greater than 33.0 is considered outlier.

Detect Outliers

```
# Values less than Lower Limit  
print(df3[df3['low'] < low_low_limit])  
  
# Values greater than Upper Limit  
print(df3[df3['low'] > low_high_limit])
```

	high	low
20	34	-10
	high	low
90	31	40

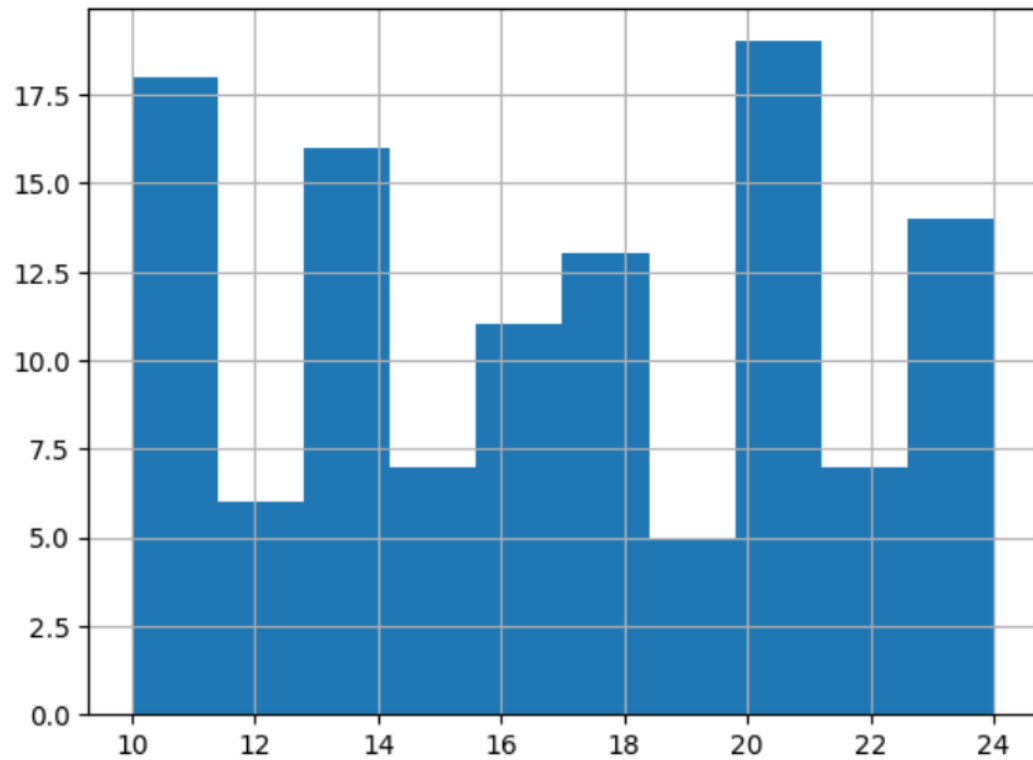
Remove Outliers

```
df3.drop(df3[df3['low'] < low_low_limit].index, inplace=True)  
df3.drop(df3[df3['low'] > low_high_limit].index, inplace=True)
```

Check Results

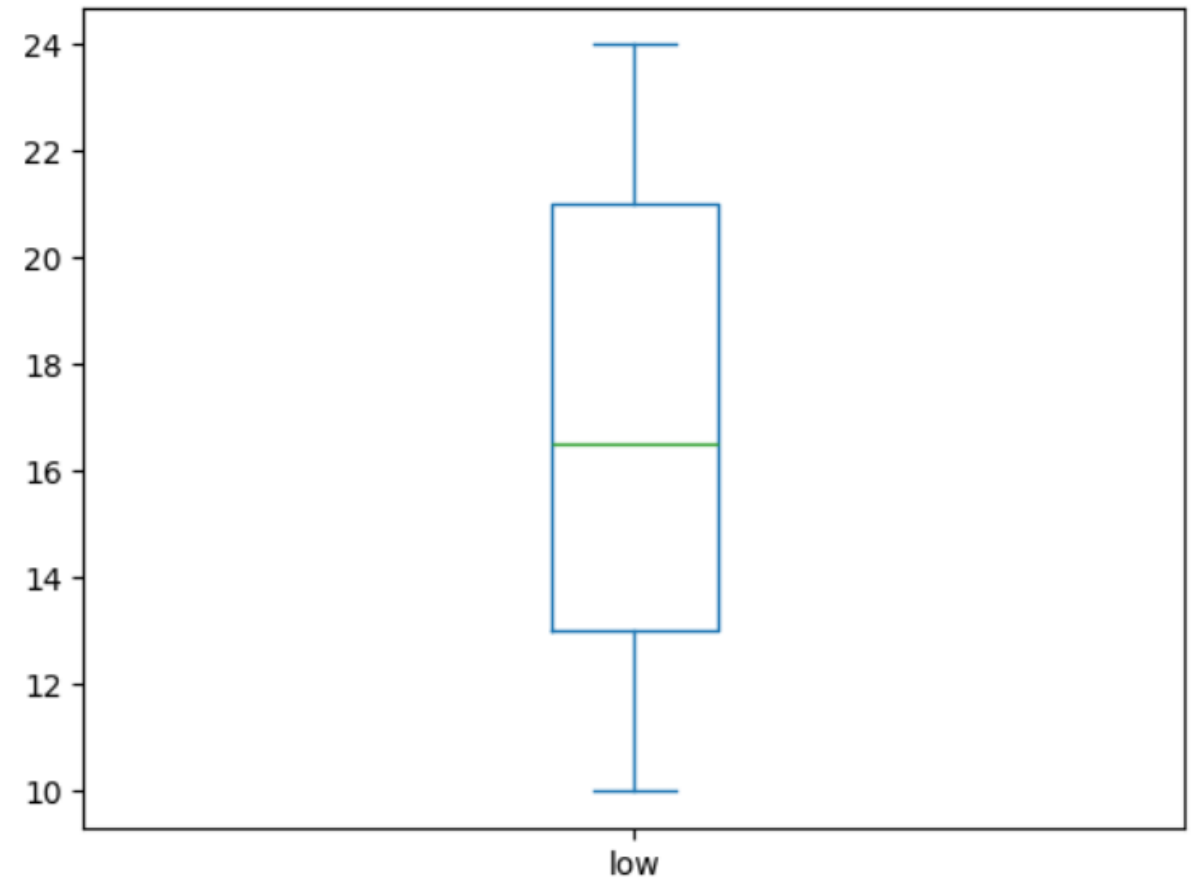
```
df3['low'].hist()
```

<Axes: >



```
df3['low'].plot(kind = 'box')
```

<Axes: >



3. DATA REDUCTION

- In the field of data analysis, dealing with large and complex datasets is a common challenge.
- Data reduction techniques offer practical solutions to handle such datasets effectively.
- This section introduces two fundamental concepts for data reduction: Dimension elimination and data sampling.
- Dimensionality reduction is a crucial technique for simplifying complex datasets by reducing the number of features or variables while retaining essential information. This concept aims to improve computational efficiency, reduce noise, and enhance the interpretability of data.
- Data sampling involves the selection of a subset of data points from a larger dataset. This approach is valuable for reducing the overall dataset size while retaining its statistical characteristics and patterns. Data sampling is particularly useful when working with extensive datasets, as it can significantly improve analysis efficiency.



3.1 Dimension Elimination

Sometimes we collect a lot of data, but not all columns are useful for analysis.

Dimension Elimination =
Removing irrelevant columns.

You have four different ways to perform Dimension Elimination:

1. Slicing columns.
2. Drop columns by position.
3. Selecting important columns.
4. Deleting unimportant columns by name.

```
df4=pd.read_csv('C:/Users/Admin/Documents/Data Mining Practical/housing.csv')  
df4.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 20640 entries, 0 to 20639  
Data columns (total 10 columns):  
#   Column                Non-Null Count  Dtype  
---  -  
0   longitude             20640 non-null  float64  
1   latitude              20640 non-null  float64  
2   housing_median_age    20640 non-null  float64  
3   total_rooms           20640 non-null  float64  
4   total_bedrooms        20433 non-null  float64  
5   population            20640 non-null  float64  
6   households             20640 non-null  float64  
7   median_income         20640 non-null  float64  
8   median_house_value    20640 non-null  float64  
9   ocean_proximity       20640 non-null  object  
dtypes: float64(9), object(1)  
memory usage: 1.6+ MB
```

Method 1: Slicing a Group of Columns

```
# We take the columns from column 2 to the last
df_sample1 = df4[df4.columns[2:]]
df_sample1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   housing_median_age    20640 non-null  float64
1   total_rooms           20640 non-null  float64
2   total_bedrooms        20433 non-null  float64
3   population            20640 non-null  float64
4   households            20640 non-null  float64
5   median_income         20640 non-null  float64
6   median_house_value    20640 non-null  float64
7   ocean_proximity       20640 non-null  object
dtypes: float64(7), object(1)
memory usage: 1.3+ MB
```

Method 2: Removing Unimportant Columns Using Drop

```
# Delete the first two columns (longitude, latitude)
df_sample2 = df4.drop(df4.columns[:2], axis=1)
df_sample2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   housing_median_age    20640 non-null  float64
1   total_rooms           20640 non-null  float64
2   total_bedrooms        20433 non-null  float64
3   population            20640 non-null  float64
4   households            20640 non-null  float64
5   median_income         20640 non-null  float64
6   median_house_value    20640 non-null  float64
7   ocean_proximity       20640 non-null  object
dtypes: float64(7), object(1)
memory usage: 1.3+ MB
```

Method 3: Selecting Only the Columns You Need

```
needed_cols = ['total_rooms', 'total_bedrooms', 'population', 'households']  
df_sample3 = df4[needed_cols]  
df_sample3.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 20640 entries, 0 to 20639  
Data columns (total 4 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0   total_rooms     20640 non-null  float64  
1   total_bedrooms  20433 non-null  float64  
2   population      20640 non-null  float64  
3   households      20640 non-null  float64  
dtypes: float64(4)  
memory usage: 645.1 KB
```

Method 4: Deleting Unimportant Columns by Name

```
dontneeded_cols = ['latitude', 'longitude', 'median_income', 'median_house_value', 'ocean_proximity']  
df_sample4 = df4.drop(dontneeded_cols, axis=1)  
df_sample4.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 20640 entries, 0 to 20639  
Data columns (total 5 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0   housing_median_age  20640 non-null  float64  
1   total_rooms        20640 non-null  float64  
2   total_bedrooms     20433 non-null  float64  
3   population         20640 non-null  float64  
4   households         20640 non-null  float64  
dtypes: float64(5)  
memory usage: 806.4 KB
```


3.2 Data Sampling (Taking Data Samples)

Sampling by Numbers (Taking a Specific Number of Rows)

`sample(n=...)` →
Specific number of
rows, no duplication by
default

```
# Taking 5 random rows
sample_by_number = df4.sample(n=5)
print(sample_by_number)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
15339	-117.31	33.18	16.0	1835.0	430.0	
19439	-121.00	37.68	15.0	1232.0	180.0	
2483	-120.36	36.14	18.0	1206.0	274.0	
16392	-121.23	38.04	32.0	1829.0	262.0	
17127	-122.13	37.47	30.0	1480.0	294.0	

	population	households	median_income	median_house_value	\
15339	599.0	399.0	2.0147	87700.0	
19439	408.0	196.0	6.9682	182400.0	
2483	622.0	217.0	1.8264	62000.0	
16392	677.0	243.0	6.1805	247900.0	
17127	1126.0	301.0	4.9830	166700.0	

	ocean_proximity
15339	NEAR OCEAN
19439	INLAND
2483	INLAND
16392	INLAND
17127	NEAR BAY

Sampling by Percentage (Taking a Percentage of Data)

`sample(frac=...)`
→ Percentage of data, no duplication by default

```
# Randomize 0.1% of data
sample_by_frac = df4.sample(frac=0.001)
print(sample_by_frac)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
16337	-121.36	38.04	9.0	2167.0	370.0	
11885	-117.37	33.98	27.0	1342.0	547.0	
5216	-118.25	33.94	43.0	793.0	NaN	
17707	-121.81	37.32	26.0	2528.0	511.0	
8754	-118.36	33.82	28.0	67.0	15.0	
16718	-120.66	35.49	17.0	4422.0	945.0	
10017	-121.21	39.24	7.0	4194.0	673.0	
191	-122.24	37.79	43.0	1626.0	376.0	
6856	-118.16	34.06	25.0	4284.0	741.0	
10538	-117.70	33.48	10.0	3458.0	638.0	
19180	-122.58	38.46	15.0	2936.0	517.0	
15540	-117.14	33.07	12.0	9302.0	1603.0	
15796	-122.42	37.76	52.0	3587.0	1030.0	
17487	-119.82	34.43	15.0	1482.0	345.0	
13956	-117.06	34.17	21.0	2520.0	582.0	
5506	-118.42	33.98	3.0	475.0	155.0	
4455	-118.17	34.09	45.0	1327.0	271.0	
4548	-118.21	34.02	52.0	22.0	7.0	
2402	-119.34	36.62	26.0	1922.0	339.0	
20424	-118.75	34.18	4.0	16704.0	2704.0	
7680	-118.10	33.92	35.0	2017.0	383.0	

Sampling with Replacement

``sample(..., replace=True)`` | Row duplication is possible in the sample

The same row may appear more than once because we have set ``replace=True``.

The number of rows required (15) is greater than the number of rows available (11).

We must use `replace=True`, so that some rows can be taken more than once to reach the required number (15).

If we set `replace=False` (which is the default), then we cannot take more than the number of rows in the DataFrame or subset without duplication.

```
# Take 15 rows out of the first 11 rows with substitution
sample_with_replacement = df4.loc[:10].sample(n=15, replace=True)
print(sample_with_replacement)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
10	-122.26	37.85	52.0	2202.0	434.0	
3	-122.25	37.85	52.0	1274.0	235.0	
7	-122.25	37.84	52.0	3104.0	687.0	
9	-122.25	37.84	52.0	3549.0	707.0	
6	-122.25	37.84	52.0	2535.0	489.0	
6	-122.25	37.84	52.0	2535.0	489.0	
8	-122.26	37.84	42.0	2555.0	665.0	
9	-122.25	37.84	52.0	3549.0	707.0	
1	-122.22	37.86	21.0	7099.0	1106.0	
4	-122.25	37.85	52.0	1627.0	280.0	
2	-122.24	37.85	52.0	1467.0	190.0	
6	-122.25	37.84	52.0	2535.0	489.0	
2	-122.24	37.85	52.0	1467.0	190.0	
4	-122.25	37.85	52.0	1627.0	280.0	
4	-122.25	37.85	52.0	1627.0	280.0	

	population	households	median_income	median_house_value	ocean_proximity
10	910.0	402.0	3.2031	281500.0	NEAR BAY
3	558.0	219.0	5.6431	341300.0	NEAR BAY
7	1157.0	647.0	3.1200	241400.0	NEAR BAY
9	1551.0	714.0	3.6912	261100.0	NEAR BAY
6	1094.0	514.0	3.6591	299200.0	NEAR BAY
6	1094.0	514.0	3.6591	299200.0	NEAR BAY
8	1206.0	595.0	2.0804	226700.0	NEAR BAY
9	1551.0	714.0	3.6912	261100.0	NEAR BAY
1	2401.0	1138.0	8.3014	358500.0	NEAR BAY
4	565.0	259.0	3.8462	342200.0	NEAR BAY
2	496.0	177.0	7.2574	352100.0	NEAR BAY
6	1094.0	514.0	3.6591	299200.0	NEAR BAY
2	496.0	177.0	7.2574	352100.0	NEAR BAY
4	565.0	259.0	3.8462	342200.0	NEAR BAY
4	565.0	259.0	3.8462	342200.0	NEAR BAY

4. DATA DISCRETIZATION AND SCALING

- In the realm of data analysis, preparing and transforming data to ensure it is suitable for analysis is a critical step.
- This section introduces two fundamental concepts for data preprocessing: Data scaling and data discretization.
- Data scaling is the process of transforming data into a consistent range to ensure that no single feature disproportionately influences an analysis. This concept is vital for algorithms that rely on distance calculations or gradient descent, as well as for visualizing data with varying scales.
- Data discretization involves the transformation of continuous data into discrete intervals or categories. This technique is beneficial for simplifying complex data, reducing noise, and making data more interpretable. Discretization can be based on statistical measures like quartiles or domain knowledge.



4.1 Data Discretization (Converting Continuous Values to Categories)

Discretize Population

Example: We divide the population into two categories, "popular" and "not popular," based on the average.

```
df5 = pd.read_csv('C:/Users/Admin/Documents/Data Mining Practical/housing.csv')
df5['population'] = df5['population'].fillna(df5['population'].mean())

population_mean = df5['population'].mean()

df5['popular'] = np.select(
    [df5['population'] < population_mean, df5['population'] >= population_mean],
    ['not popular', 'popular'],
    default='unknown'
)

print(df5['popular'].value_counts())
```

popular	
not popular	13128
popular	7512
Name: count, dtype: int64	

Discretize Rooms

Example: We classify the residences according to total_rooms and total_bedrooms into four categories:

LL → Few rooms and few bedrooms
HH → Many rooms and many bedrooms
LH → Few rooms but many bedrooms
HL → Many rooms but few bedrooms
Medium → Other cases

```
conditions = [  
    (df5['total_rooms'] < df5['total_rooms'].quantile(0.25)) & (df5['total_bedrooms'] < df5['total_bedrooms'].quantile(0.25)),  
    (df5['total_rooms'] > df5['total_rooms'].quantile(0.75)) & (df5['total_bedrooms'] > df5['total_bedrooms'].quantile(0.75)),  
    (df5['total_rooms'] < df5['total_rooms'].quantile(0.5)) & (df5['total_bedrooms'] > df5['total_bedrooms'].quantile(0.5)),  
    (df5['total_rooms'] > df5['total_rooms'].quantile(0.5)) & (df5['total_bedrooms'] < df5['total_bedrooms'].quantile(0.5)),  
]  
  
values = ['LL', 'HH', 'LH', 'HL']  
  
df5['rooms'] = np.select(conditions, values, default='Medium')  
  
print(df5['rooms'].value_counts())
```

```
rooms  
Medium    9839  
LL         4105  
HH         4081  
HL         1310  
LH         1305  
Name: count, dtype: int64
```

4.2 Data Scaling (Adjusting the numerical data values so that they are on the same scale or within a specific range)

- **Min-Max Scaling (Normalization):**
- We convert all values to a fixed range, usually [0,1].
- The smallest value becomes 0, and the largest value becomes 1.
- Formula:

$$v'_i = \frac{v_i - \min(v)}{\max(v) - \min(v)}$$

```
df5['population_MinMax'] = (df5['population'] - df5['population'].min()) / (df5['population'].max() - df5['population'].min())  
df5['population_MinMax']
```

```
0      0.008941  
1      0.067210  
2      0.013818  
3      0.015555  
4      0.015752
```

```
...
```

```
20635    0.023599  
20636    0.009894  
20637    0.028140  
20638    0.020684  
20639    0.038790
```

```
Name: population_MinMax, Length: 20640, dtype: float64
```

```
df5['population_MinMax'].describe()
```

```
count    20640.000000  
mean      0.039869  
std       0.031740  
min       0.000000  
25%      0.021974  
50%      0.032596  
75%      0.048264  
max       1.000000
```

```
Name: population_MinMax, dtype: float64
```


- **Z-Score Standardization:**
- We convert the values so that the mean = 0 and the standard deviation = 1.
- Each value represents the number of standard deviations from the mean.
- Formula:

$$v'_i = \frac{v_i - \text{mean}}{\text{std}}$$

```
df5['population_Z'] = (df5['population'] - df5['population'].mean()) / df5['population'].std()
df5['population_Z']
```

```
0      -0.974405
1       0.861418
2     -0.820757
3     -0.766010
4     -0.759828
...
20635  -0.512579
20636  -0.944382
20637  -0.369528
20638  -0.604415
20639  -0.033976
Name: population_Z, Length: 20640, dtype: float64
```

```
df5['population_Z'].describe()
```

```
count      2.064000e+04
mean      -1.514723e-17
std         1.000000e+00
min       -1.256092e+00
25%       -5.637952e-01
50%       -2.291262e-01
75%        2.644885e-01
max        3.024960e+01
Name: population_Z, dtype: float64
```


- **Decimal Scaling:**
- we divide each value by 10 to the power of t to reduce the size of the number, but the ratios between the values remain the same.
- This is useful if we have very large numbers and want to make them smaller and easier to work with.
- The formula:

$$v'_i = \frac{v_i}{10^t}$$

- $v'_i \rightarrow$ Value after scaling
- $v_i \rightarrow$ Original value
- $t \rightarrow$ Number of digits of the absolute largest value in the column + 1

```
max_val = df5['population'].max()
print(max_val)
```

```
35682.0
```

```
t = len(str(int(max_val))) + 1
print(t)
```

```
6
```

```
df5['population_decimal'] = df5['population'] / (10**t)
df5['population_decimal']
```

```
0      0.000322
1      0.002401
2      0.000496
3      0.000558
4      0.000565
```

```
...
```

```
20635  0.000845
20636  0.000356
20637  0.001007
20638  0.000741
20639  0.001387
```

```
Name: population_decimal, Length: 20640, dtype: float64
```

```
print(df5['population_decimal'].describe())
```

```
count    20640.000000
mean      0.001425
std       0.001132
min       0.000003
25%       0.000787
50%       0.001166
75%       0.001725
max       0.035682
```

```
Name: population_decimal, dtype: float64
```

Summary

Data Visualization	• Scatter Plots : Relationship between two quantitative variables • Line Plots : Change over time • Area Plots : Change and size of values • Bar Charts : Comparing categories • Histograms : Distribution of data • Box Plot : Distribution + Outliers • Pie Charts : Percentages of categories • Color Map : Relationship + a third variable
Data Preprocessing	• DEALING WITH MISSING VALUES : Dropping Missing Values , Filling with Constant, Filling with ffill , Filling with bfill , Filling with mean , Filling with mode • DEALING WITH OUTLIERS : Detect Outliers Using IQR • DATA REDUCTION : Dimension elimination and data sampling • DATA DISCRETIZATION : Converting Continuous Values to Categories • DATA SCALING : Min-Max Scaling , Z-Score Standardization, Decimal Scaling

Task

- Each student downloads a dataset (CSV, Excel, JSON...) and applies the following steps:
- Display `head()`, `info()` and `describe()`.
- Handle missing values.
- Detect and handle outliers.
- Reduce dimensions if needed.
- Apply discretization for numeric columns.
- Apply scaling techniques.
- Visualize relationships using appropriate plots.



Thank You

